arXiv:2512.09060v3 [stat.CO] 30 Aug 2026

All Emulators are Wrong, Many are Useful, and Some are More Useful Than Others: A Reproducible Comparison of Computer Model Surrogates

Kellin N. Rumsey[1]
Statistical Sciences
Los Alamos National Laboratory
Los Alamos, NM 87545

Graham C. Gibson
Statistical Sciences
Los Alamos National Laboratory
Los Alamos, NM 87545

Devin Francom
Statistical Sciences
Los Alamos National Laboratory
Los Alamos, NM 87545

Reid Morris
Statistical Sciences
Los Alamos National Laboratory
Northeastern University

Abstract

Accurate and efficient surrogate modeling is essential for modern computational science, and there are a staggering number of emulation methods to choose from. With new methods being developed all the time, comparing the relative strengths and weaknesses of different methods remains a challenge due to inconsistent benchmarking practices and (sometimes) limited reproducibility and transparency. In this work, we present a large-scale, fully reproducible comparison of 32 distinct emulators across 60 canonical test functions and 40 real emulation datasets. To facilitate rigorous, apples-to-apples comparisons, we introduce the R package `duqling`, which streamlines reproducible simulation studies using a consistent, simple syntax, and automatic internal scaling of inputs. This framework allows researchers to compare emulators in a unified environment and makes it possible to replicate or extend previous studies with minimal effort, even across different publications. Our results provide detailed empirical insight into the strengths and weaknesses of state-of-the-art emulators and offer guidance for both method developers and practitioners selecting a surrogate for new data. We discuss best practices for emulator comparison and highlight how `duqling` can accelerate research in emulator design and application.

1 Introduction

A *computer model emulator* (also called a *surrogate* or *meta-model*) is a fast-to-evaluate statistical proxy for an expensive (and/or proprietary) computer model. Emulators are essential tools in uncertainty quantification (UQ), enabling analyses that would otherwise be computationally prohibitive or infeasible.

According to Lu et al. [2024], there are roughly ten thousand new emulation-related papers released each year, with more than two hundred thousand papers written on the subject since the year 2000. We have recently witnessed the emergence of countless surveys, review papers, and even textbooks (Razavi et al. [2012], Sudret et al. [2017], Alizadeh et al. [2020], Liu et al. [2020], Wang et al. [2022], Gramacy [2020], Sauer et al. [2023a] to name a few). When it comes to choosing an emulation method, there are dozens of possible methods, each with its own extensions and variants. Hence, it is difficult to know for certain which needle in the emulation-haystack one ought to select for a particular task. Developers of new surrogate models are expected to compare their methods against existing approaches, yet the sheer number of possible comparisons often undermines the interpretability and trustworthiness of these results. Practitioners in various communities are seldom aware of the pros and cons (or indeed, the existence) of techniques used by neighboring scientific communities. McClarren et al. [2011], Salter and Williamson [2016], Laloy and Jacques [2019], Heaton et al. [2019], Hutchings et al. [2023], Chipman et al. [2012], Rumsey et al. [2023] and Collins et al. [2024] represent a few (among many) recent examples of emulation papers that include a large-scale "bake-off" style comparison. Similar benchmarking efforts have also appeared in related areas, including hyperparameter optimization, supervised learning, and classification [Cowen-Rivers et al., 2022, Fernández-Delgado et al., 2014, Kigerl et al., 2022, Marrs and Francom, 2025]. Not surprisingly, different papers come to different conclusions depending on the context.

In this work, we focus on the basic scenario where computer models have a vector of inputs and scalar response. Our primary goals are:

1) We present a framework for reproducible and transparent simulation studies, which allows for a direct comparison of emulation methods, even across papers.

2) We perform a massive comparison of 32 popular methods using a diverse suite of 60 test functions and 40 datasets.

3) We offer guidance and tools for helping practitioners make informed choices about which emulator(s) they should consider for a particular UQ task.

4) We offer guidance and tools to help emulator methodology developers understand strengths and weaknesses (and opportunities for improvement) of various emulators.

The remainder of this paper is organized as follows. In Section 2, we survey some popular approaches for emulation and discuss the 32 methods that are the focus of the present work. In Section 3, we present a framework for reproducible emulator comparisons, facilitated by the `duqling` R package. Results of this comparison are given in Section 4 with further analysis and discussion given in Section 5.

2 Review of Emulation Approaches: Meet the Contestants

Deciding on the 32 emulators to present in this paper was no small task. We sought to create a diverse set of approaches used across many of the various specialized UQ communities, with an emphasis on methods that provide full predictive distributions rather than point estimates. Accordingly, our primary performance metric is continuous ranked probability score (CRPS; see Equation (1)), a proper scoring rule for probabilistic predictions that directly evaluates this distributional output. Inevitably, many strong alternatives were left out; one has to draw the line somewhere.

The Gaussian process (GP) is considered by many to be the "gold-standard" in surrogate modeling, and is thus a good place to start (along with some popular variants). Because GP computation scales cubically with the size of the training data, GPs can become impractical for even moderately sized datasets. For this reason, approximate Gaussian process regression has been a vibrant area of research over the past two decades (see Liu et al. [2020] for an extensive review), and so we include a diverse set of representatives here. Basis function regressors are a broad and popular alternative to GPs (though the GP itself can technically be included in this class), including linear models [Lindley and Smith, 1972], polynomial chaos expansions [Wiener, 1938], multivariate adaptive regression splines [Friedman, 1991], and many more [Denison et al., 2002]. Finally, we include many methods popular in the machine learning community. Although we make an attempt to categorize the results below, we note that these groupings are intended as a useful heuristic and not a definitive classification. Most methods described below could fit into multiple categories.

Uncertainty estimation is handled in a variety of ways throughout this study. Most commonly, this is done by equipping a method with a probability model and using statistical inference to quantify uncertainty. Bayesian methods are often preferred, both for their well-calibrated uncertainty and their reduced sensitivity to tuning parameter choices. Alongside these, we also include methods based on variational inference, conformal inference and the bootstrap, which offer alternative perspectives on uncertainty quantification.

Emulator	Full Name	Reference	R Package
Gaussian Processes and Their Variants			
GP	Gaussian Process	Rasmussen and Williams [2006]	`hetGP`
RGASP	Robust Gaussian Process	Gu et al. [2019]	`RobustGaSP`
GPYTORCH	Gaussian Process (Python)	Gardner et al. [2018]	`gpytorch`*
DEEPGP	Deep Gaussian Process	Sauer et al. [2023c]	`deepgp`
TREEGP	Treed Gaussian Process	Braun et al. [2024]	`GPTreeO`
HETGP	Heteroskedastic GP	Binois et al. [2018]	`hetGP`
Approximate Gaussian Processes			
MPGP	Matching Pursuit GP	Keerthi and Chu [2005]	`knrumsey /spareGParts`
LAGP	Local Approximate GP	Gramacy and Apley [2015]	`laGP`
ALCGP	Active Learning Cohn GP	Gramacy and Apley [2015]	`laGP`
BCMGP	Bayesian Committee Machine	Deisenroth and Ng [2015]	`knrumsey /spareGParts`
RFFGP	Random Fourier Features GP	Rahimi and Recht [2007]	`gplite`
FITCGP	Inducing Point GP (FITC)	Snelson and Ghahramani [2005]	`gplite`
SVIGP	Sparse Variational GP	Hensman et al. [2013]	`gpytorch`*
SVECGP	Scaled Vecchia GP	Katzfuss et al. [2022]	`knrumsey /spareGParts`
Basis Function Regression			
BLM	Bayesian Linear Model	Gelman et al. [2013]	`stats`
BART	Bayes. Additive Regression Trees	Chipman et al. [2012]	`BART`
BASS	Bayes. Adaptive Spline Surfaces	Francom et al. [2018]	`BASS`
TBASS	T-distributed BASS	Rumsey et al. [2024a]	`GBASS`
QBASS	Quantile (Median) BASS	Rumsey et al. [2024a]	`GBASS`
BPPR	Bayes. Projection Pursuit Reg.	Collins et al. [2024]	`gqcollins /BayesPPR`
SPCE	Sparse Bayes. Polynomial Chaos	Shao et al. [2017]	`khaos`
APCE	Adaptive Bayes. Polynomial Chaos	Rumsey et al. [2026]	`khaos`
Machine Learning and Miscellaneous			
BNN	Bayesian Neural Network	Neal [2012]	`bnns`
GBC	Generative Bayesian Computation	Polson and Sokolov [2026]	`VadimSokolov /gbc-surrogate`*
RVM	Relevance Vector Machine	Tipping [1999]	`knrumsey /spareGParts`
BOOTRF	Random Forest with Bootstrap	Breiman [2001]	`randomForest`
CONFRF	Conformal Random Forest	Johansson et al. [2014]	`conforest`
NGBOOST	Natural Gradient Boosting	Duan et al. [2020]	`Akai01 /ngboost`*
BCART	Bayesian CART	Chipman et al. [1998]	`tgp`
BTREELM	Treed Bayesian Linear Model	Chipman et al. [2002]	`tgp`
BLASSO	Bayesian LASSO	Park and Casella [2008]	`BayesianLasso`

Table 1: Summary of emulators and surrogate models compared in this study. Packages available on CRAN are listed by name. For methods not available on CRAN, a GitHub repository is indicated by `username/repo` or simply `/repo` if hosted at `https://github.com/knrumsey`. The symbol * denotes implementations that rely on external Python libraries, accessed via the `reticulate` package, rather than native R packages [Ushey et al., 2025]. In several cases, dedicated repositories were created to provide a convenient interface to methods that are otherwise unavailable in R.

This study is not intended to provide *de facto* answers about which emulator is best. There is no free lunch (see Section 5 for more), and the search for a one-size-fits-all approach is misguided. Many excellent emulation approaches are not considered here, and it is not possible to test the full space of possible computer models (despite our best efforts). For this reason, we evaluate each method using its "out-of-the-box" settings when possible, making sensible choices (balancing speed and accuracy) based on the literature and software documentation when necessary.

It is important to note that, with sufficient tuning, the results for any given method could change drastically. Indeed, many of the emulators we consider are universal approximators, capable of emulating any function arbitrarily well given enough training data. With finite data, however, their inference algorithms can yield varying degrees of over- or under-fitting, which practitioners must try to control through tuning.

Finally, given the behavioral range of the test functions, even the best implementations can occasionally fail. When this happens, we include a series of fallback models to ensure that every function can still be evaluated. We have made every effort to be reasonable in these choices, but the reported performance of each emulator may still be influenced by how often it fails. The R code used to test each method is provided in the supplement. The full list of emulators is given in Table 1, along with a reference and the associated software package.

2.1 Gaussian Processes and Their Variants

Gaussian Process (GP, RGASP, GPYTORCH): Gaussian processes have long been at the forefront of computer model emulation. In fact, some authors (historically) have used the terms GP and emulator interchangeably. See Rasmussen and Williams [2006] or Gramacy [2020] (or with high probability, a randomly selected reference in this paper) for details.

Though there are many options for Gaussian process modeling in R, we include two state-of-the-art implementations. The first (GP) is the `mleHomGP()` function from the `hetGP` package [Binois and Gramacy, 2021], which is known for its speed and stability. The second (RGASP) is the `rgasp()` function from the `RobustGaSP` package [Gu et al., 2019], which offers a sophisticated framework for Gaussian process parameter estimation, along with strong theoretical guarantees Gu et al. [2018], but is comparatively less optimized for speed.

We also include a Python-based implementation (GPYTORCH) using the `gpytorch` library (accessed through `reticulate`) which provides a modern framework for scalable Gaussian process modeling. In

our experiments, this implementation emphasizes computational efficiency but tends to sacrifice some predictive accuracy relative to the R-based implementations [Gardner et al., 2018, Ushey et al., 2025]. All three implementations estimate the nugget, rather than fixing it at zero, so that models can appropriately adapt to non-deterministic settings; for RGASP, we set `nugget.est = TRUE`.

Deep GP (DEEPGP): Deep GPs (DGPs), first introduced by Damianou and Lawrence [2013], are better equipped to handle nonstationarity and complex response surfaces by stacking multiple latent GP layers, enabling automatic adaptation to regime changes and heterogeneity in the data. We use the `deepgp` R package to fit a two-layer DGP, leveraging elliptical slice sampling and a Vecchia approximation for efficient Bayesian inference [Sauer et al., 2023c,b].

Treed GP (TREEGP): Treed Gaussian processes (TGPs), introduced by Gramacy and Lee [2008], partition the input space with a tree structure and fit independent GPs within each region. TGPs are particularly useful for emulating computer models that exhibit discontinuities, abrupt regime shifts, or other forms of nonstationarity. We use the `GPTreeO` R package, which provides an efficient implementation of a TGP surrogate [Braun et al., 2024].

Heteroskedastic GP (HETGP): Heteroskedastic Gaussian processes (HetGPs) extend traditional GPs by modeling not only the mean response but also input-dependent noise variance [Binois et al., 2018]. This enables accurate uncertainty quantification even when the data exhibit varying levels of noise across the input space. We use the `hetGP::mleHetGP()` function, which offers fast and reliable estimation [Binois and Gramacy, 2021].

2.2 Approximate Gaussian Processes

Matching Pursuit (MPGP): One simple version of approximate GP regression is known as a subset of data (SoD) approach, in which training samples are discarded until the number of remaining samples is feasible for likelihood evaluation. Selecting a subset of points uniformly at random is typically cheap but ineffective. The Matching Pursuit GP (MPGP) of Keerthi and Chu [2005] constructs the subset sequentially, greedily adding a data point that leads to a large increase in the log posterior of the full GP model (see also Smola and Bartlett [2000]).

We use the implementation in the `spareGParts` R package (`github.com/knrumsey/spareGParts`). Because the MPGP algorithm requires fixed kernel hyperparameters, we apply a simple two-step loop:

alternating between subset selection and kernel re-estimation. By default, this loop runs twice before returning the final model.

Local Approximate GP (LAGP): The local approximate GP (laGP) is similar to a SoD approach, except that the sub-design is custom-built for each new prediction point [Gramacy and Apley, 2015]. This allows for good performance with a much smaller GP compared to global SoD approaches and naturally accounts for nonstationarity in the computer model response. Neighborhood subsets are greedily selected to minimize an empirical Bayesian mean square prediction error, improving over the simpler nearest neighbor construction [Cressie, 1990] without drastically increasing the complexity. We use the `laGP` R package with neighborhood sizes equal to $\min(\max(30, \lfloor\sqrt{n}\rfloor), 100)$ after initializing neighborhoods with the 10 nearest neighbors [Gramacy, 2016]. As shown in Section 5.2, LAGP can be sensitive to the nugget specification. We initialize the nugget at $\mathtt{g} = \max(10^{-7}\text{Var}(y), 10^{-9})$.

Active Learning Cohn (ALCGP): Another global SoD approach using the `laGP` package. This functionality (described as "beta" in the documentation) uses a similar active learning strategy to construct a single neighborhood for all prediction points jointly. We use a subset size of $\min(\max(100, 2\lfloor\sqrt{n}\rfloor), 300, n-1)$.

Bayesian Committee Machine (BCMGP): The Bayesian Committee Machine (BCM) of Tresp [2000] makes GP regressions scalable by partitioning the data into a collection of manageable subsets. A GP, called an "expert", is fit to each subset, sharing hyperparameters by maximizing the product of individual likelihoods. Unlike in SoD, all of the experts are aggregated during prediction. Our implementation in `spareGPparts` uses the robust BCM of Deisenroth and Ng [2015], which weights the experts based on how far they are from the query location. A fast partitioning around medoids (PAM) algorithm is used to choose the $\lfloor\sqrt{n}/2\rfloor$ data partitions.

Random Fourier Features (RFFGP): The random features approach of Rahimi and Recht [2007] maps the data into a randomized low-dimensional feature space, where inner products approximate those of a shift-invariant kernel. This can also be viewed as a sparse basis function regression model. We use the `gplite` R package with $\min(512, 2\lfloor\sqrt{n}\rfloor)$ basis functions.

Inducing Point GP (FITCGP, SVIGP): Inducing point methods are a widely used class of scalable Gaussian process approximations. Like SoD methods, these approximations use a reduced set of "inducing points" (also called pseudo-inputs) to improve efficiency; however, these inducing points need not belong to the training data and can be freely optimized, offering greater flexibility.

The fully independent training conditional (FITC) method [Snelson and Ghahramani, 2005] is a seminal inducing point approach that defines a sparse approximation to the GP covariance based on the inducing set, and then maximizes the resulting (approximate) marginal likelihood using gradient descent. We use the `gplite` R package with $\min(\max(100, 2\lfloor\sqrt{n}\rfloor), 300, n-1)$ inducing points.

We also include a stochastic variational inducing point (SVIGP) following Hensman et al. [2013], which instead introduces a variational distribution over the inducing variables. This formulation leads to an evidence lower bound (ELBO) that decomposes over the original data points, enabling the use of mini-batch stochastic gradient descent for efficient optimization of all hyperparameters. Our implementation uses the `gpytorch` library in Python, accessed through `reticulate`.

Scaled Vecchia GP (SVECGP): Vecchia approximations are a powerful tool for approximating complex high-dimensional likelihoods by decomposing the joint density into a sequence of low-dimensional conditional distributions [Vecchia, 1988]. This approach creates a sparse representation of the model leading to tractable inference for large datasets that would otherwise be computationally prohibitive. In the context of GPs, Stein et al. [2004] first demonstrated how Vecchia approximations could scale up spatial GP inference and these ideas have been refined and extended by Katzfuss et al. [2020] and Katzfuss and Guinness [2021].

The scaled Vecchia GP used in this paper is described by Katzfuss et al. [2022] and is tailored towards computer models by transforming the input space via Fisher scoring. The implementation we use is available in the `spareGParts` R package, which primarily serves as a wrapper for the `GpGp` and `GPvecchia` packages [Guinness et al., 2024, Katzfuss et al., 2024].

2.3 Basis Function Regression

Bayesian Linear Model (BLM): The Bayesian linear model (BLM) serves as a simple reference point, fitting a linear regression with conjugate Bayesian inference for coefficients and noise variance. See, e.g., Lindley and Smith [1972], Gelman et al. [2006].

Bayesian Additive Regression Trees (BART): BART is a popular and widely-used ensemble of regression trees, similar to boosted trees [Friedman, 2001], but equipped with a probabilistic framework that enables uncertainty quantification [Chipman et al., 2012]. It employs a Bayesian back-fitting MCMC algorithm to sample trees conditionally on the rest of the forest. Prior distributions are placed on tree structure and terminal node parameters, with regularization to control tree depth and prevent overfitting. We use the `wbart()` function from the `BART` R package using default settings.

Bayesian Adaptive Spline Surfaces (BASS): The Multivariate Adaptive Regression Splines (MARS™) algorithm was originally proposed by Friedman [1991] as a fast and effective way to greedily construct basis functions for spline surfaces from the bottom up (as opposed to top-down or regularization-based approaches). Denison et al. [1998a] introduced the first Bayesian extension to MARS (BMARS), which was subsequently refined by Nott et al. [2005] and Francom and Sansó [2020]. BMARS employs a simplified form of reversible jump Markov chain Monte Carlo (RJMCMC; Green [1995]) to adaptively learn the number of basis functions, providing principled uncertainty quantification out of the box. We use the `BASS` R package (named to avoid trademark issues) using default settings.

Generalized BASS (TBASS, QBASS): Generalized BMARS extends BMARS/BASS by replacing the Gaussian error assumption with the broader class of generalized hyperbolic distributions [Rumsey et al., 2024a]. This framework includes several robust alternatives to standard BASS. We consider two such variants: `GBASS::tbass()`, which uses a t-distributed error model, and `GBASS::qbass()`, which targets conditional quantiles. For TBASS, we use 3 degrees of freedom; for QBASS, we set $q = 0.5$, corresponding to median regression. The latter induces an inflated Laplace predictive distribution rather than a Gaussian one, which may be advantageous for robust point prediction but less well aligned with scoring rules such as CRPS in the simulation settings considered here.

Bayesian Projection Pursuit Regression (BPPR): Projection pursuit regression (PPR; Friedman and Stuetzle [1981]) approximates complex multivariate functions as sums of flexible ridge functions applied to linear projections of the inputs, with single index models as a simplified case [Ichimura, 1987]. The Bayesian version, recently developed by Collins et al. [2024], provides UQ and uses RJMCMC to infer both the number and structure of the ridge functions. We use the `BayesPPR` R package (`github.com/gqcollins/BayesPPR`) with default settings.

Sparse Bayesian Polynomial Chaos (SPCE): Polynomial chaos expansions (PCE) have a long history as a popular emulator in certain UQ sub-communities [Wiener, 1938, Xiu and Karniadakis, 2002]. PCE expresses the model output as a sum of tensor products of orthogonal polynomials. The approach of Shao et al. [2017] performs efficient Bayesian inference on sparse PCE models with fixed basis functions. Model selection is performed via a ranking procedure coupled with an information criterion. Initially the model is heavily constrained, and the set of candidate basis functions is enriched until a sufficiently expressive model can be found. The `khaos` R package provides an implementation of the Shao et al. [2017] algorithm with a few minor deviations, including a different enrichment strategy [Rumsey et al., 2026].

Adaptive Bayesian Polynomial Chaos (APCE): Adaptive Bayesian PCE employs a modified BASS algorithm [Francom and Sansó, 2020] with tensor-product Legendre polynomials as the basis and a new prior/proposal to better learn input interactions [Rumsey et al., 2026]. We use the implementation in the `khaos` R package.

2.4 Machine Learning and Miscellaneous Methods

Bayesian Neural Network (BNN): As neural networks and their variants become increasingly popular, it is no surprise that they are being used as emulators for UQ [Sun and Wang, 2019, Tripathy and Bilionis, 2018]. Traditional NNs often provide poor UQ, so we focus here on Bayesian neural networks (BNNs), which offer a principled probabilistic framework for UQ [Neal, 2012]. Our implementation uses the `bnns` R package [Chatterjee, 2025], which is built on Stan [Carpenter et al., 2017], and we fit a BNN with two hidden layers with 8 nodes each.

Generative Bayesian Computation (GBC): Generative Bayesian Computation (GBC), recently proposed by Polson and Sokolov [2026], is a likelihood-free surrogate modeling approach that builds on ideas from implicit quantile networks (IQNs) [Dabney et al., 2018]. Rather than specifying an explicit probabilistic model, GBC learns the conditional quantile function of the response given the inputs, allowing predictive samples to be generated by evaluating the model at randomly drawn quantile levels. This yields a flexible, scalable neural network-based surrogate that does not rely on assumptions of Gaussianity or stationarity. We use the reference implementation available at `github.com/VadimSokolov/gbc-surrogate`, interfaced through `reticulate`. The model is trained using default settings, and predictive samples are obtained by repeatedly sampling from the learned quantile function.

Relevance Vector Machine (RVM): The relevance vector machine (RVM), proposed by Tipping [1999], is designed as a probabilistic extension of the support vector machine [Cortes and Vapnik, 1995], with the added advantage of typically producing sparser representations. RVMs build predictions as weighted sums of kernel functions, but use (empirical) Bayesian inference to account for uncertainty. Our implementation follows Tipping's original formulation, with an added discrete prior over the lengthscale parameter to guard against poor kernel choices. The code is available in the `spareGParts` R package.

Bootstrapped Random Forest (BOOTRF): Random forests (RF) are a popular machine learning (ML) technique [Breiman, 2001], and their strong predictive performance makes them an obvious candidate for inclusion in the present work. However, standard RFs do not directly provide UQ, so special care must

be taken (see also the discussion on BART). It is sometimes (dubiously) claimed that the spread of the weak-learner predictions in boosting ensembles characterizes uncertainty in a meaningful way; we take an approach here which is similar in spirit, but slightly more principled. Specifically, we train a "super-forest" of 100 RFs using the nonparametric bootstrap [Efron, 2000], resampling the data for each fit. The collection of predictions from these models is then interpreted (for better or for worse) as a predictive distribution. We use the `randomForest` R package for implementation.

Conformal Random Forest (CONFRF): Conformal inference (CI) provides a principled framework for constructing valid predictive intervals, ensuring coverage at any predefined confidence level [Vovk et al., 2005]. CI will be used here to provide UQ for random forests. Specifically, we use the RFoK framework discussed by Johansson et al. [2014], which efficiently leverages out-of-bag estimates from the random forest for conformal calibration, removing the need for a separate validation set. Our implementation is in the `conforest` R package (`github.com/knrumsey/conforest`) which uses `randomForest` under the hood.

Natural Gradient Boosting (NGBOOST): Natural Gradient Boosting (NGBoost) extends gradient boosting machines, like XGBoost [Chen and Guestrin, 2016], to allow for probabilistic predictions. By using natural gradients and flexible multiparameter loss functions, NGBoost can fit a wide variety of outcome distributions and provides meaningful UQ [Duan et al., 2020]. It can be paired with any base learner and any continuous parametric family; in this study, we opt for a Gaussian distribution and decision trees as the base learner. We use the R package located at `Akai01/ngboost` which calls the Python `ngboost` library via `reticulate` [Ushey et al., 2025].

Bayesian CART (BCART): Bayesian CART (BCART) is a fully Bayesian version of the classic Classification and Regression Trees (CART) algorithm of Breiman et al. [2017], enabling probabilistic uncertainty quantification in tree-based models. Two foundational BCART algorithms were proposed independently in 1998 [Chipman et al., 1998, Denison et al., 1998b], each introducing a Bayesian framework for recursive partitioning and inference on trees. We use the implementation from the `tgp` R package, which follows the approach of Chipman et al. [1998].

Bayesian Treed Linear Model (BTREELM): The Bayesian treed linear model (BTREELM), introduced in Chipman et al. [2002], extends the BCART framework by placing a linear model at each of the terminal nodes rather than a constant mean. This hybrid approach combines the interpretability of trees with the flexibility

of local linear modeling, often improving predictive accuracy while retaining coherent Bayesian inference. It is also implemented in the `tgp` package.

Bayesian LASSO (BLASSO): The Bayesian LASSO introduces a Laplace prior on regression coefficients, yielding a fully Bayesian extension of the widely used LASSO regularization method [Tibshirani, 1996]. The Bayesian formulation of Park and Casella [2008] enables rigorous uncertainty quantification and typically requires less tuning of penalty parameters. We use the implementation in the `BayesianLasso` R package [Ormerod et al., 2025].

Baseline Model (BASELINE): This model estimates the mean and variance of the response variable (independent of the inputs) and makes predictions from the resulting Gaussian distribution. This model is used as a baseline for comparison to identify scenarios where more sophisticated emulators are prone to overfitting.

3 The `duqling` Package

Fair and reproducible comparison of emulation methods across different settings is challenging. The `duqling` framework was developed to address this need by providing a standardized and extensible platform for running large-scale emulator comparisons, enabling results to be meaningfully compared, even across papers.

To illustrate how the framework works in practice, the following R code snippet shows how a user could compare a new emulator against those used in this paper. While the details of the simulation study will be introduced in the next section, the example below gives a preview of how easily such comparisons can be made:

```
library(duqling)

# Run simulation study
my_results <- run_sim_study(
                 fit_shiny_new_emulator,   # User-defined
                 pred_shiny_new_emulator,  # User-defined
                 fnames=get_paper_funcs(), # List of test functions
                 n_train=1000,             # Training set size
```

```
                    NSR=c(0, 0.1),            # Noise-to-signal ratio
                    design_type="LHS",        # Design matrix generation
                    replications=10,          # Number of replications
                    mc_cores=10)              # Number of cores to use
```

The function `run_sim_study()` executes a full simulation study by applying a user-defined emulator to a configurable set of test functions, design settings, noise levels, and replications. The design type can be set to (i) Latin hypercube sampling (LHS; [McKay et al., 1979]), (ii) independent uniform sampling, (iii) a uniform grid, or (iv) user-specified custom designs. The noise-to-signal ratio (NSR) is defined as the ratio of the observational noise variance to the unconditional variance of the simulator response, i.e., $NSR = \mathrm{Var}(\epsilon)/\mathrm{Var}(f(\boldsymbol{x}))$.

After running the simulation study, any of the figures in Section 4 can be created with just a few lines of code, as shown below.

```
# Load the paper data
data("sim_study_testfuncs")

# Process and combine simulation studies
duq_paper <- process_sim_study(sim_study_testfuncs)
duq_myres <- process_sim_study(my_results)
duq_joined <- join_sim_study(duq_paper, duq_myres)

# Filter for specific case
duq_joined_filtered <- filter_sim_study(duq_joined, n_train=1000, NSR=0)

# Make figures and analysis
summarize_sim_study(duq_joined_filtered)
rankplot_sim_study(duq_joined_filtered, metric="CRPS")
```

A Python version of the package is also available at `https://github.com/reidmorris/duqling_py`, though it is less actively maintained than the R implementation. Alternatively, the R package can be accessed from Python using interoperability tools such as `rpy2` [Gautier, 2026].

3.1 Test Functions

The `duqling` package includes a large (and growing) collection of test functions. This paper focuses on scalar-output, deterministic functions with continuous inputs, and the automated simulation study framework currently supports only this setting. The package also includes test functions representing stochastic simulators, functions with categorical inputs, and multivariate or functional outputs, although automated simulation studies for these cases are still in development.

The set of test functions used here was curated to reflect a broad range of modeling challenges. These include functions with input dimensions ranging from 1 to 100; both stationary and nonstationary behavior; smooth and discontinuous surfaces; low and high effective dimension; and varying degrees of linearity and nonlinearity. Several test functions include fully inert variables or are constant-valued, which helps test for overfitting.

All 60 test functions are listed in a table in the supplement, along with the input dimension and some statistical summaries. References and additional details for each function are provided in the `duqling` package documentation; many are also described in greater detail in the Simon Fraser virtual library of simulation experiments [Surjanovic and Bingham, 2013].

3.2 Data Sets

We also evaluate emulator performance on 40 real-world datasets, 19 of which are publicly available in the `duqling` package, with the remaining 21 not yet approved for release. The datasets span a wide range of applications, including problems in chemistry, materials science, nuclear physics, climate modeling, epidemiology, and plume dispersion. Sample sizes range from 138 to $22{,}229$, and input dimensionality ranges from 2 to 178. In some cases, inputs were selected using formal design of experiments, while in others they represent posterior draws from a calibration procedure [Kennedy and O'Hagan, 2001] and may exhibit strong correlation.

For analysis and visualization, we divide the datasets into two groups based on training size: small datasets with $n < 2000$ and large datasets with $n \geq 2000$. A full list of datasets, along with their sample sizes, input dimensions, and references, is provided in Table 2.

Table 2: Summary of real-world datasets used in this study. The number of observations is n and the number of inputs is p.

Dataset	n	p	`duqling`	Reference
plate_deformation	138	7	No	Francom et al. [2018]
wind_speed	200	17	No	Edmunds et al. [2013]
strontium_plume_p4b	300	20	Yes	Volkova et al. [2008]
strontium_plume_p104	300	20	Yes	Volkova et al. [2008]
spectra1	500	4	No	Klein et al. [2020]
spectra2	500	4	No	Klein et al. [2020]
pbx9501_gold	500	6	Yes	Rumsey et al. [2025]
pbx9501_ss304	500	6	Yes	Rumsey et al. [2025]
pbx9501_nickel	500	6	Yes	Rumsey et al. [2025]
pbx9501_uranium	500	6	Yes	Rumsey et al. [2025]
ptw1	500	10	No	IMPALA Team [2025]
discoflux_flyer	914	6	No	Francom et al. [2021]
taylor_foot	962	11	No	Biswas et al. [2021]
taylor_length	962	11	No	Biswas et al. [2021]
jc	1,000	5	No	IMPALA Team [2025]
flyerPTW1	1,000	11	No	Biswas et al. [2021]
flyerPTW2	1,000	11	No	Biswas et al. [2021]
flyer_plate104	1,000	11	Yes	Walters et al. [2018]
diffusion_1D	1,000	62	No	Hlobilová et al. [2024b]
fair_climate_ssp1-2.6_year2200	1,001	45	Yes	Smith et al. [2018]
fair_climate_ssp2-4.5_year2200	1,001	45	Yes	Smith et al. [2018]
fair_climate_ssp3-7.0_year2200	1,001	45	Yes	Smith et al. [2018]
taylor_cylinder1	1,438	8	No	Sjue et al. [2023]
stochastic_sir	2,000	4	Yes	Rumsey et al. [2024a]
diffusion_2D	2,000	53	No	Hlobilová et al. [2024a]
acme_climate	2,980	66	No	Sargsyan et al. [2015]
ptw2	3,701	13	No	IMPALA Team [2025]
SLOSH_low	4,000	5	Yes	Hutchings et al. [2023]
SLOSH_mid	4,000	5	Yes	Hutchings et al. [2023]
SLOSH_high	4,000	5	Yes	Hutchings et al. [2023]
Z_machine_max_vel1	5,000	6	Yes	Brown and Hund [2018]
Z_machine_max_vel2	5,000	6	Yes	Brown and Hund [2018]
Z_machine_max_vel_all	5,000	30	Yes	Brown and Hund [2018]
e3sm_mnar	9,122	2	Yes	Grosskopf et al. [2021]
nuclear_data	9,307	178	No	Francom et al. [2019b]
e3sm_mcar	10,000	2	Yes	Grosskopf et al. [2021]
icf1	10,000	5	No	Gaffney et al. [2020]
icf2	10,000	5	No	Gaffney et al. [2020]
diablo_canyon_plume	18,000	13	No	Francom et al. [2019a]
okc_plume	22,229	5	No	Brown et al. [2009]

3.3 Reproducible Simulation Studies

A central design goal of `duqling::run_sim_study()` is strict reproducibility. For any combination of test function, training size, design type, noise level, and replication index, the generated training data—both inputs and responses—are deterministic. This holds even when simulations are run in different orders or batches. For example:

```
emulators <- get_emulator_functions(c("rgasp", "lagp", "bart"))
results1 <- run_sim_study(emulators$fit_func, emulators$pred_func,
                          fnames = c("borehole", "ishigami"),
                          NSR = 0,
                          replications = 1:10)

results2 <- run_sim_study(emulators$fit_func, emulators$pred_func,
                          fnames = "ishigami",
                          NSR = 0,
                          replications = c(1, 7))
```

The rows corresponding to `ishigami`, `NSR = 0`, and the specified replications will be identical in both cases (except for timing, which is machine-dependent). Internally, each simulation scenario is transformed into a unique seed via a polynomial hashing scheme, ensuring reproducibility across platforms and workflows.

For real-world datasets, `run_sim_study_data()` provides analogous functionality, using dataset name, fold number, and cross-validation type (both cross-validation and bootstrap are currently supported) as the hash keys. Full details of the hashing algorithm are provided in the supplement.

3.3.1 Fallback Model

To maintain consistency across such a large and diverse test suite, `duqling::run_sim_study()` includes a fallback model that activates when an emulator fails during fitting or prediction. The fallback is a simple null model that predicts from a Student's t distribution fit to the marginal distribution of the training responses. While deliberately naive, it ensures that performance metrics remain well-defined even when an emulator breaks. Failures are recorded in the output via a categorical `failure_type` field, which takes the values `"none"`, `"fit"`, or `"pred"`.

The continuous ranked probability score (CRPS) serves as our primary performance metric (see Equation (1)). For a unit-variance response, the baseline model has an expected CRPS of $\pi^{-1/2} \approx 0.564$, which we use as a reference point after rescaling. Additional details are provided in Section 4.

3.3.2 Simulation Output Format

The function `run_sim_study()` returns an R data frame where each row corresponds to a single emulator applied to a specific simulation scenario and replication. Key columns include:

- `method`, `fname`, `n_train`, `NSR`, `design_type`, and `replication`, identifying the simulation configuration.
- `RMSE`, `FVU`, and `CRPS` as primary performance metrics, where `RMSE` is root mean squared error, `FVU` is the fraction of variance unexplained (see Section SM5.1 for definition), and `CRPS` is the continuous ranked probability score (see Equation (1)).
- `t_fit`, `t_pred`, and `t_tot`, for fit time, prediction time, and total runtime in seconds, respectively.

Additional columns can optionally be included when we seek to further assess predictive uncertainty, including empirical coverage (`COVER<level>`), mean interval length (`LENGTH<level>`), mean interval score (`MIS<level>`), and summary statistics of CRPS across test points. A complete description is provided in the supplement.

When $NSR > 0$, `duqling` distinguishes between latent and noisy test evaluations. By default, point-prediction metrics (RMSE, MAE, Bias, and FVU) are computed against the latent simulator response, whereas distributional and interval-based metrics (including CRPS and coverage) are computed against the noisy test response.

The function `run_sim_study_data()` returns results in the same format, but replaces `fname` and `replication` with `dname`, `fold`, and `fold_size`, reflecting its use of cross-validation. The full simulation results used in this paper are bundled with the `duqling` package and can be accessed via `data(sim_study_testfuncs)` and `data(sim_study_realdata)`.

4 Results

In this section, we compare the results of 32 emulators across 60 synthetic test functions and 40 real-world simulation datasets. Given the scope of the study, the analysis here is necessarily high-level, but in Sec-

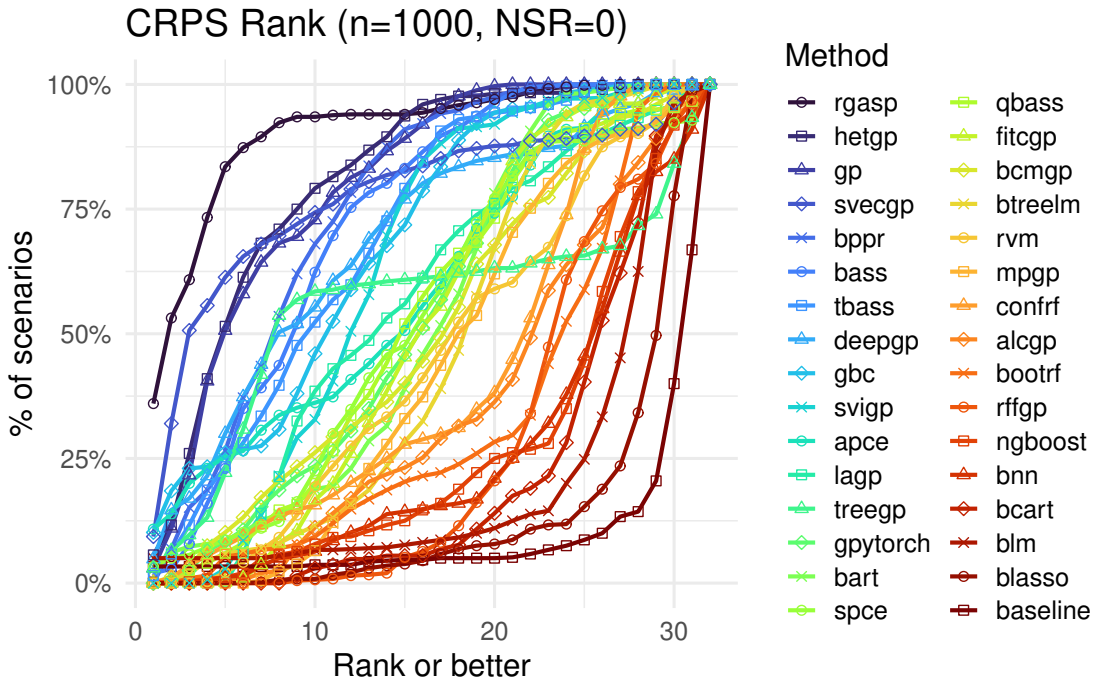


Figure 1: Cumulative rank-plot for $n = 1000$, $NSR = 0$ setting. The curve for each emulator shows the proportion of cases that the method was at least top r out of 32 in terms of CRPS, for $r = 1, \ldots, 32$.

tion 5 we demonstrate how `duqling` can be used to conduct more granular, targeted investigations. All performance metrics are computed on out-of-sample test sets. For synthetic test functions, we use maximin Latin hypercube designs with $n_{\text{test}} = 1000$ to assess out-of-sample performance [McKay et al., 1979]. For the real-world datasets, we apply 10-fold cross-validation, so n_{test} is determined by the size of the dataset. Our primary performance metric is continuous ranked probability score (CRPS), a proper scoring rule for probabilistic predictions that evaluates both predictive accuracy and uncertainty quantification simultaneously. CRPS is widely regarded as a gold-standard metric for evaluating predictions made with uncertainty [Gneiting and Raftery, 2007]. For $i = 1, 2, \ldots, n_{\text{test}}$

$$\text{CRPS}_i = \text{CRPS}(y_{\text{test},i}, \{\hat{y}_1, \ldots, \hat{y}_M\}) = \frac{1}{M}\sum_{j=1}^{M} |\hat{y}_j - y_{\text{test},i}| - \frac{1}{M(M-1)}\sum_{j<k} |\hat{y}_j - \hat{y}_k|. \tag{1}$$

where $\hat{y}_1, \ldots, \hat{y}_M$ are predictive samples from the emulator for the ith test point. Metrics based solely on point predictions, such as FVU (see Section SM5.1 for definition), are also available but do not assess uncertainty quantification. Alternative diagnostics, including empirical coverage and interval-based metrics,

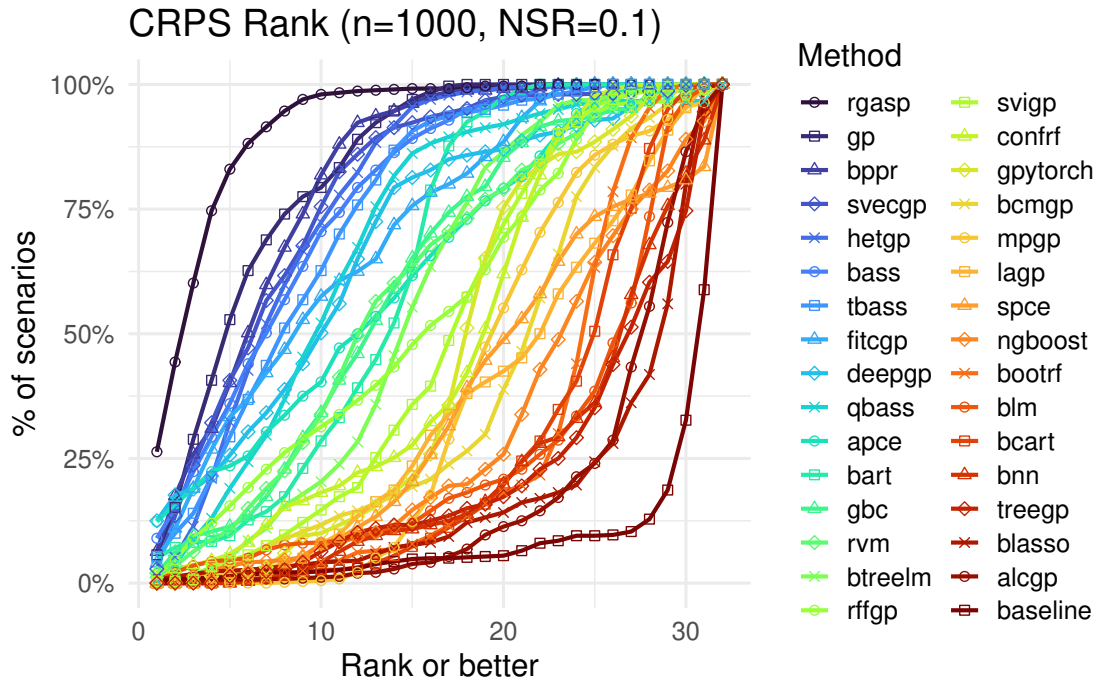


Figure 2: Cumulative rank-plot for $n = 1000$, $NSR = 0.1$ setting. The curve for each emulator shows the proportion of cases that the method was at least top r out of 32 in terms of CRPS, for $r = 1, \ldots, 32$.

can also be computed within the `duqling` framework (see Section SM5.4), but are not the primary focus of this study.

To facilitate interpretation and comparability across datasets, we rescale all CRPS values as if the true response had unit variance. On this scale, a CRPS value of $1/\sqrt{\pi} \approx 0.564$ corresponds to the expected score of the baseline model, which is useful for interpretation. In practice, the difference between sufficiently small CRPS values is largely irrelevant beyond a certain point. This is particularly true in many primary applications of emulators, such as sensitivity analysis, optimization, and calibration. Sensitivity analysis techniques are often robust to small to moderate emulator errors (though overfitting presents more of a challenge). The degree to which calibration and optimization are affected depends on the relative size of measurement and model discrepancy uncertainties. Thus, while many of our analyses report rankings and show differences on this sensitive scale, we note that all CRPS values below a certain threshold (e.g., 0.001) are likely to be indistinguishable for many applications.

There are many ways to summarize the n_{test} CRPS values computed for each emulator on each test set, each highlighting a different aspect of emulator performance. In the analyses that follow, each visualization

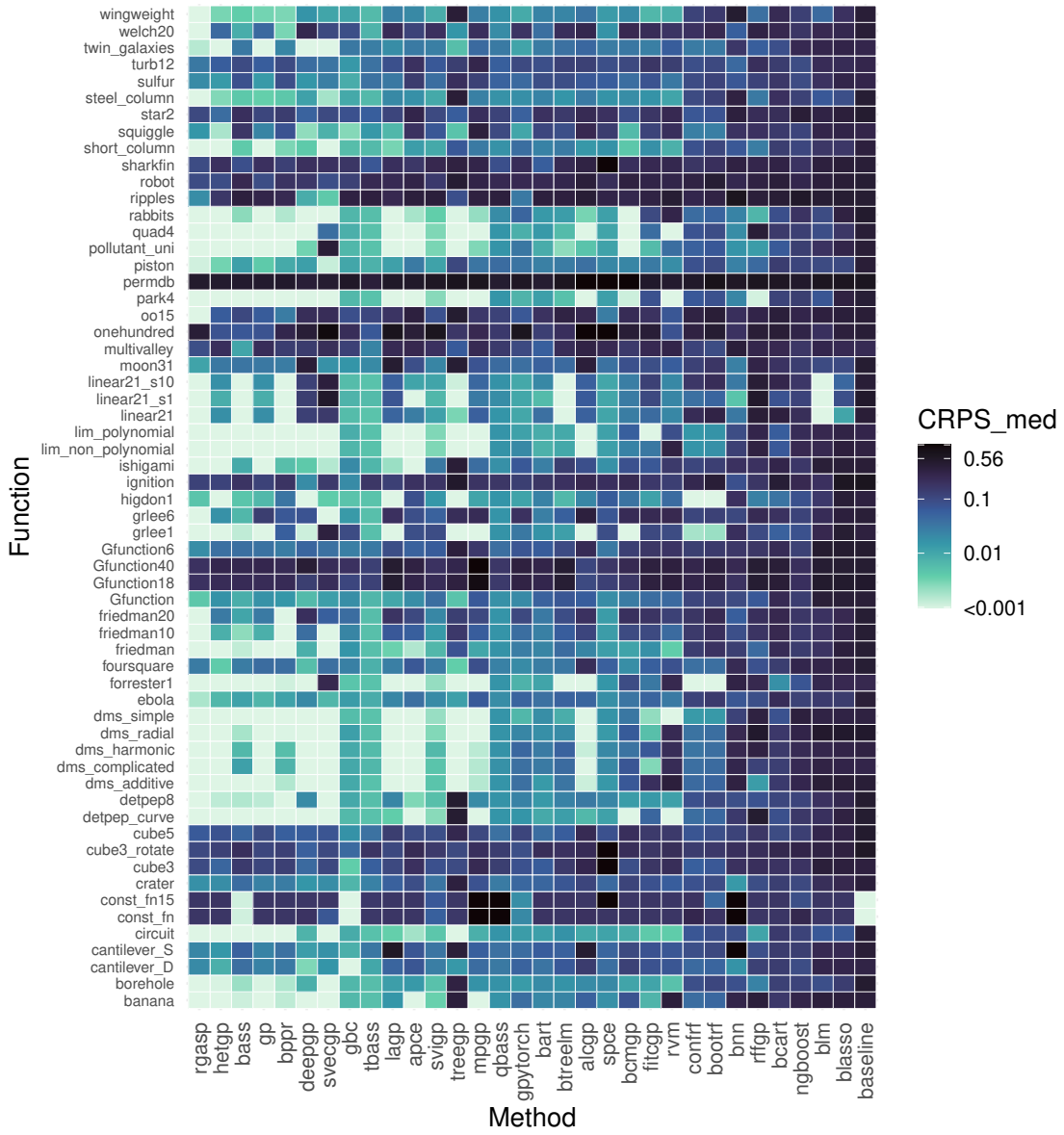


Figure 3: A heatmap of CRPS (truncated between 0.001 and 1.0 for visual simplicity) for the synthetic test functions in the $n = 1000, NSR = 0$ setting.

is based on a different perspective: cumulative rank plots rank emulators based on mean CRPS, heatmaps display median CRPS (over the n_{test} test points) averaged across replications, and Pareto front plots compare methods using a relative mean CRPS measure. Specifically, for emulator i and simulation scenario j, we

define

$$t_{i,j} = \frac{\text{CRPS}_{i,j} + \omega}{\min_k [\text{CRPS}_{k,j}] + \omega}, \tag{2}$$

which measures performance relative to the best-performing emulator in each scenario, where $\omega \geq 0$ is a small smoothing parameter, and $\text{CRPS}_{i,j}$ is the mean across the n_{test} test points. Choosing a non-zero ω indicates that values of CRPS much smaller than ω should all be considered equally good, from a pragmatic perspective. For Pareto front plots, the value $\omega = 0.001$ is used, and the $t_{i,j}$ are capped at 1000 to reduce the influence of extreme outliers on the overall results.

For the real-world datasets, we also make use of performance profile plots [Dolan and Moré, 2002, Moré and Wild, 2009], which provide an alternative view of relative performance across datasets. The performance profile for emulator i is given by

$$\rho_i(\tau) = \frac{1}{J} \sum_{j=1}^{J} \mathbb{1}(t_{i,j} \leq \tau),$$

which represents the proportion of the J simulation scenarios for which emulator i is within a factor τ of the best-performing method (we use $\omega = 0$ here). In practice, we find performance profiles particularly useful for the real datasets, where the number of scenarios is smaller and practical differences are of primary interest. For the synthetic test functions, however, the wide range of behaviors and larger number of scenarios make performance profiles less readable and interpretable, so we rely primarily on rank-based summaries in that setting.

With such a large and rich dataset, there are dozens of plausible analyses one could conduct, each with its own variants, and the resulting story can vary significantly depending on the perspective taken. In what follows, we present a range of viewpoints, with many additional analyses and figures provided in the supplement. For the sake of brevity, however, some choices had to be made. We encourage interested readers to explore their own analyses using the tools and data provided. All plots are generated using the `duqling` package (see supplement for details).

Simulation Settings. We consider the following benchmark settings:

- $n = 1000$, $NSR = 0$, 10 replications, 60 test functions.
- $n = 1000$, $NSR = 0.1$, 10 replications, 60 test functions.
- $n = 500$, $NSR = 0$, 10 replications, 60 test functions. All emulators.

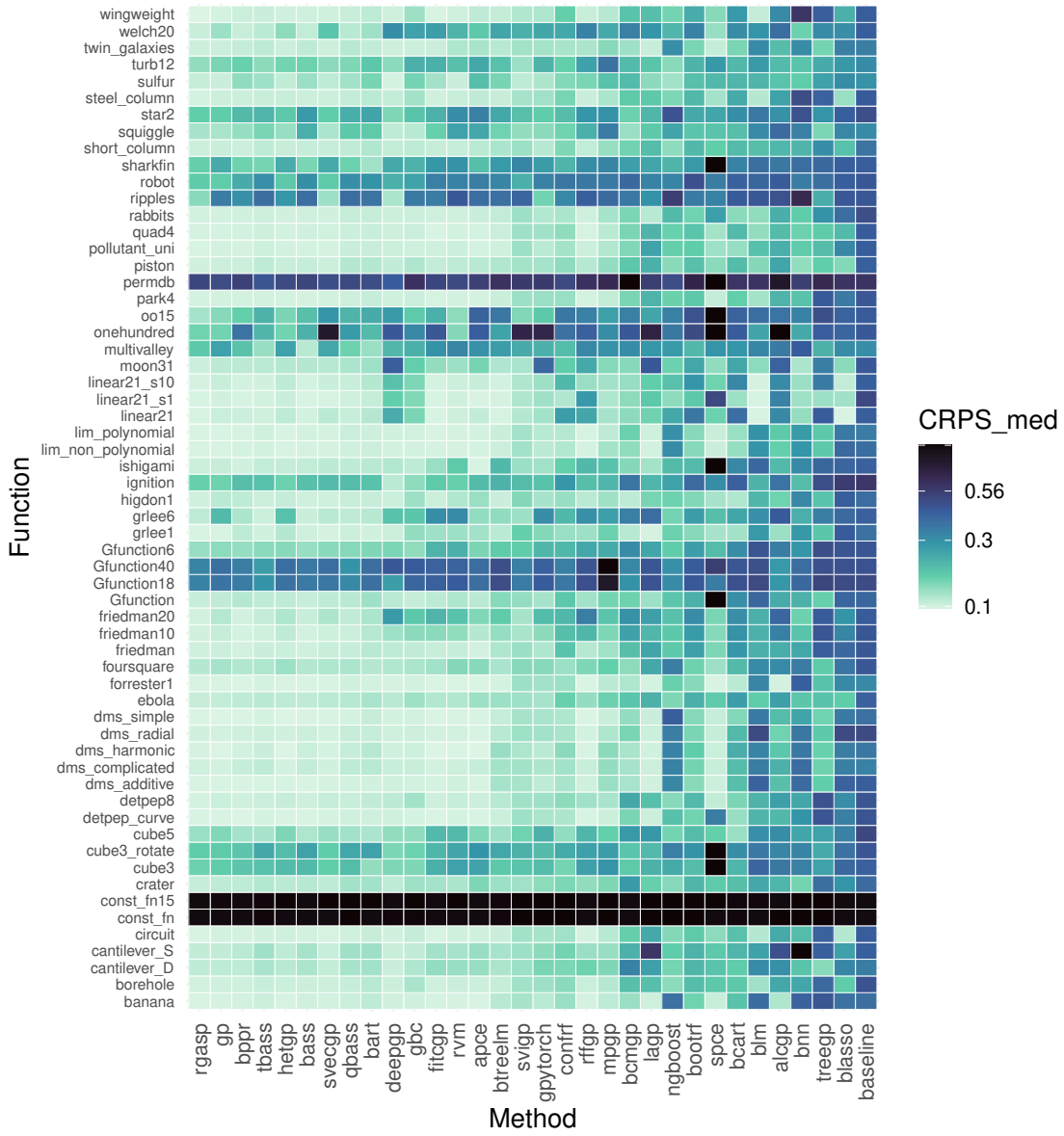


Figure 4: A heatmap of CRPS (truncated between 0.001 and 1.0 for visual simplicity) for the synthetic test functions in the $n = 1000, NSR = 0.1$ setting.

- $n = 500$, $NSR = 0.1$, 10 replications, 60 test functions. All emulators (see supplement).
- $n = 5000$, NSR = 0, 10 replications, 60 test functions. Excluded: GP, RGASP, DEEPGP, HETGP.
- 21 real datasets with $n < 2000$. All emulators.

- 19 real datasets with $n \geq 2000$. Excluded: GP, RGASP, DEEPGP, HETGP.

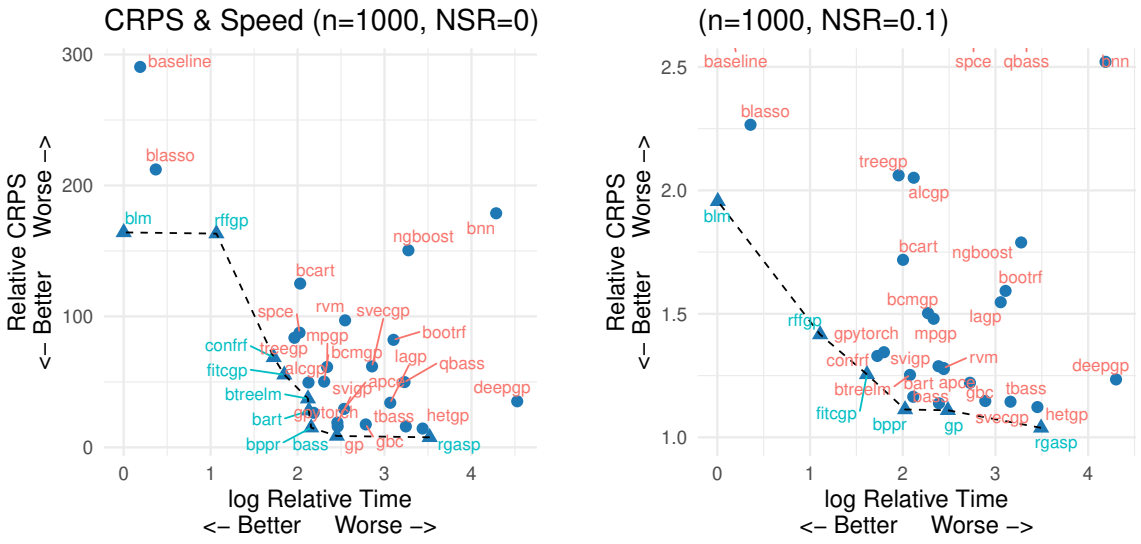
In the computer model emulation setting, the deterministic case ($NSR = 0$) is often the primary focus, but setting $NSR = 0.1$ allows us to check for overfitting, simulate more complex dynamics (some of which manifest as chaos or near stochastic behavior), or introduce light stochasticity which is sometimes of interest [Baker et al., 2022].

Caveats and Practical Considerations. We emphasize caution in interpreting these results. While we made an effort to use reasonable defaults for all emulators, these results reflect the practical implementation of each emulator as much as the underlying modeling approach. Performance might change considerably with sufficient tuning or different hyperparameter choices. We encourage readers to modify settings using `duqling` to explore these effects directly.

For computational reasons, emulators were excluded from a given setting if all simulation scenarios took more than 10 hours.

Across all methods, we use out-of-the-box fitting routines as much as possible and do not assess convergence of any of the algorithms. This is especially relevant for the GPyTorch-based methods, whose hyperparameters are estimated by iterative gradient-based optimization. While this allows these methods to complete in larger-data settings, predictive accuracy may suffer when the optimizer has not fully converged and could potentially improve with additional optimization.

Finally, although `duqling` separately reports training and prediction time, we report only the total runtime here. In some cases, these phases are not separable due to an emulator's structure or implementation. This may lead to misleading impressions for some methods (e.g., SVECGP, LAGP) that train quickly but predict more slowly, but in other contexts, omitting prediction time could be equally misleading. For example, in applications requiring MCMC (e.g., model calibration) fast prediction is often critically important and cannot be parallelized; see Rumsey et al. [2023] for discussion. We also note that runtime comparisons should be interpreted with caution, as computational performance can vary substantially across systems. The experiments in this study were run on a Linux-based HPC cluster using 10 cores per task, on nodes with a mix of Intel and AMD processors (x86_64 architecture). As a result, the timing results reported here are most useful for relative comparisons within this study rather than as absolute benchmarks across papers or computing environments.



(a) (b)

Figure 5: Pareto front of emulators based on average relative CRPS and log relative runtime across all simulation scenarios for the $NSR = 0$ and $NSR = 0.1$ settings ($n = 1000$). Relative CRPS is computed as $(\text{CRPS} + \omega)/(\min \text{CRPS} + \omega)$ with $\omega = 0.001$ for each scenario, capped at 1000 to reduce sensitivity to outliers. The dashed line and triangle markers indicate the methods which are not dominated (slower and less accurate) by any other emulator.

4.1 Test Functions (n=1000)

In our experience, the case where approximately $n = 1000$ model evaluations are available for training is both common and interesting. For many computer models, this is a pragmatic upper bound on how many times the true simulator can be run given typical high performance computing (HPC) resources. It is also the point where the computational burden of a standard Gaussian process becomes non-trivial, and many practitioners will start to consider faster, more scalable alternatives.

The $(n = 1000, NSR = 0)$ and $(n = 1000, NSR = 0.1)$ cases are summarized, respectively, by Figures 1 and 2 using cumulative rank plots across the 600 simulation scenarios (60 test functions repeated 10 times each). Curves that are concave downward represent methods that frequently outperform others and rarely rank near the bottom (e.g., RGASP in Figure 1), while concave up (e.g., BASELINE) usually rank near the bottom. An "S"-shaped curve suggests a robust method that rarely ranks at the top or bottom (e.g., MPGP), while a reverse "S" shape (e.g., TREEGP in Figure 1) indicates a method with highly variable performance across scenarios. Legend order and color assignments reflect the area under each curve, equivalent to average rank.

Gaussian process emulators tend to perform quite well, with RGASP placing as the top ranked emulator for a large number of simulation scenarios. Among the "fast" emulators, SVECGP and BPPR appear to be competitive across the various cases.

Figures 3 and 4 extend this analysis by presenting results function by function, using numerical median CRPS values (averaged across replications) rather than ranks. The emulator labels on the x-axis of these plots are ordered from left to right by the median CRPS averaged across all test functions. While the rank plots highlight how often one emulator outperforms another, the heatmaps provide insight into the magnitude of those differences broken out by test function.

To more explicitly examine the trade-off between predictive accuracy and computational cost, Figure 5 shows a Pareto front plot of average relative CRPS versus runtime. Relative CRPS is defined as described earlier, with a small offset to reduce sensitivity to negligible differences and a cap to limit the influence of outliers. Emulators near the lower-left corner of the plot represent favorable accuracy–efficiency tradeoffs, as no other method is both faster and more accurate on average across scenarios.

Several emulators not yet discussed become interesting under the lens of this tradeoff. For example, the RFFGP, CONFRF, and even BLM emulators may be worth a closer look for some problems, when computational efficiency is paramount. We also observe this trade-off within the standard GP implementations: RGASP tends to be more accurate than GP, but it is generally slower.

Together, these figures illustrate both how often one emulator outperforms another and by how much. The rank plots capture frequency of dominance, but not magnitude; the heatmaps and Pareto fronts help fill in that missing context. Additional figures using variations on these metrics are given in the supplement. For deeper scenario-level analysis, including boxplots and case studies, see Section 5.

4.2 Small Training Set (n = 500)

The case where only $n = 500$ simulator evaluations are available is common when dealing with expensive models or limited computational budgets. In Figure 6, we show cumulative rank plots summarizing emulator performance in this regime, restricted to the noise-free setting (NSR $= 0$). A full set of results, including heatmaps, timing comparisons, and the NSR $= 0.1$ case, can be found in the supplemental materials.

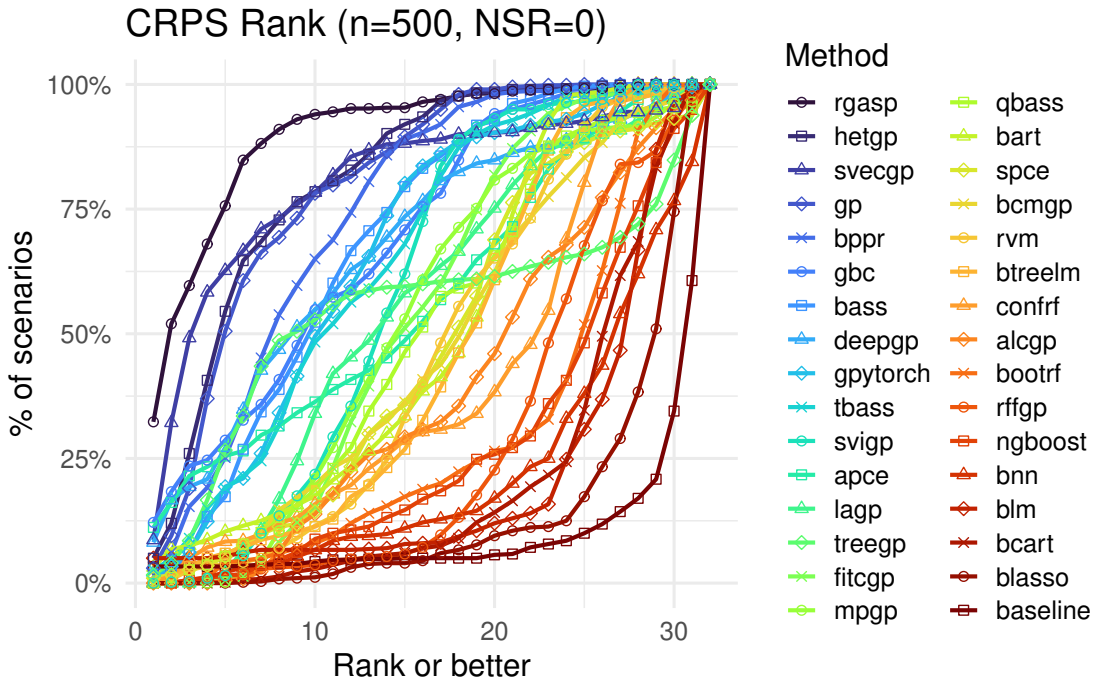


Figure 6: Cumulative rank-plot for $n = 500$, $NSR = 0$ setting. The curve for each emulator shows the proportion of cases that the method was at least top r out of 32 in terms of CRPS, for $r = 1, \ldots, 32$.

4.3 Large Training Set (n = 5000)

When $n = 5000$ evaluations are available, the design space is typically better-explored and computational constraints become more prominent. Figure 7 provides a high-level overview of emulator rankings in the noise-free setting. Due to runtime costs and practical relevance, we omit the noisy case here; additional figures for this regime are available in the supplement.

4.4 Real Data

For the real datasets included in our analysis, Figure 8 and Figure 9 show performance profiles rather than rankplots (see discussion in Section 4 and SM5.1.2 of the supplement for details). The rank-based summaries can be viewed in Section SM5.2.1 of the supplement. We remind the reader that these results reflect the implementation and hyperparameter choices for each emulator as much as the underlying approach. The tradeoff between accuracy and computation can often be explicitly controlled (e.g., the size of the conditioning sets for SVECGP or the number of training iterations for GPYTORCH). We therefore emphasize again that the comparisons here reflect these choices, rather than fully tuned performance.

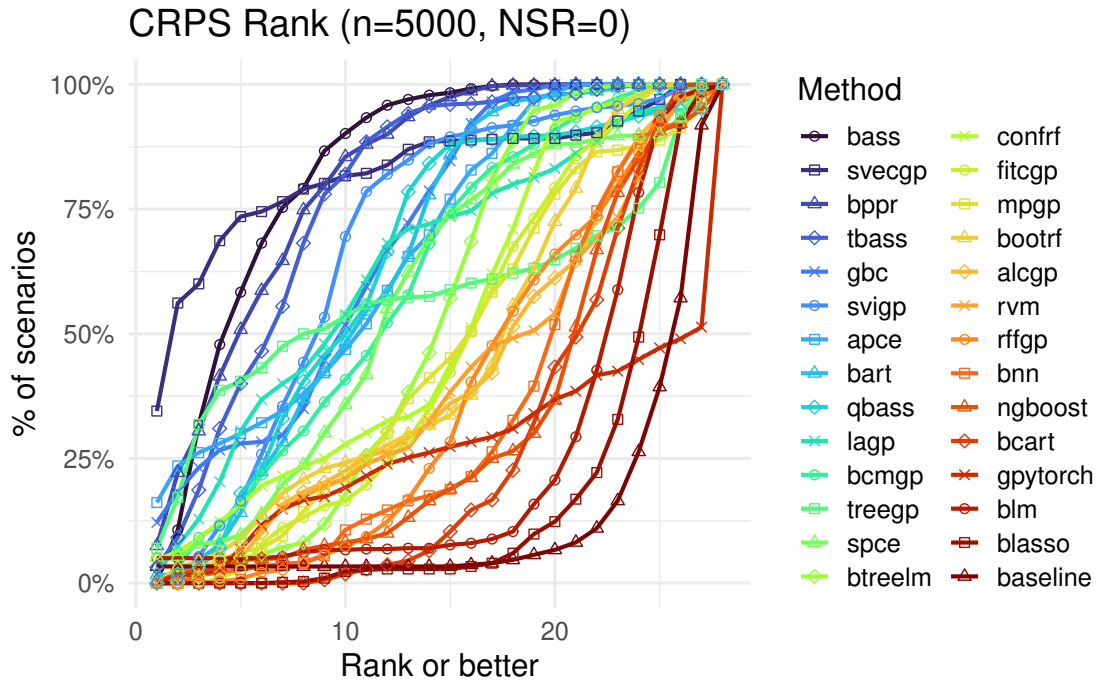


Figure 7: Cumulative rank-plot for $n = 5000$, $NSR = 0$ setting. The curve for each emulator shows the proportion of cases that the method was at least top r out of 32 in terms of CRPS, for $r = 1, \ldots, 32$.

For these computer models, the SVECGP stands out as a highly effective emulator. Across all datasets (comparing to only the fast emulators), SVECGP achieves the smallest mean CRPS in 45.0% of all folds. Had we ranked by median CRPS or the max-CRPS (across the test set), the win rate drops to 34.0% and 36.0%, respectively; still indicating strong performance, but less conclusively. And, of course, these figures do not tell us how *much* better it is in those cases, or whether the difference is practically meaningful. For instance, the relative CRPS in Figure 10b paints SVECGP as somewhat less effective (in the big data setting), likely driven by its poor performance on a few datasets, like the high-dimensional dataset $\mathrm{nuclear_data}$. On the other hand, an emulator like BASS, which demonstrates the best performance just 3.25% of the time, seems to perform reasonably well across all datasets, ranking in the top half of emulators 97.8% of the time, more than any other method. The relative mean CRPS (with $\omega = 0.001$; see Equation (2)) for BASS is 1.8, which is substantially better than even SVECGP with a relative mean CRPS of 4.9. At a minimum, this suggests that BASS is a robust and effective algorithm at its default settings; it is quite likely that with sophisticated tuning, many of the other methods could demonstrate similar proficiency across the board.

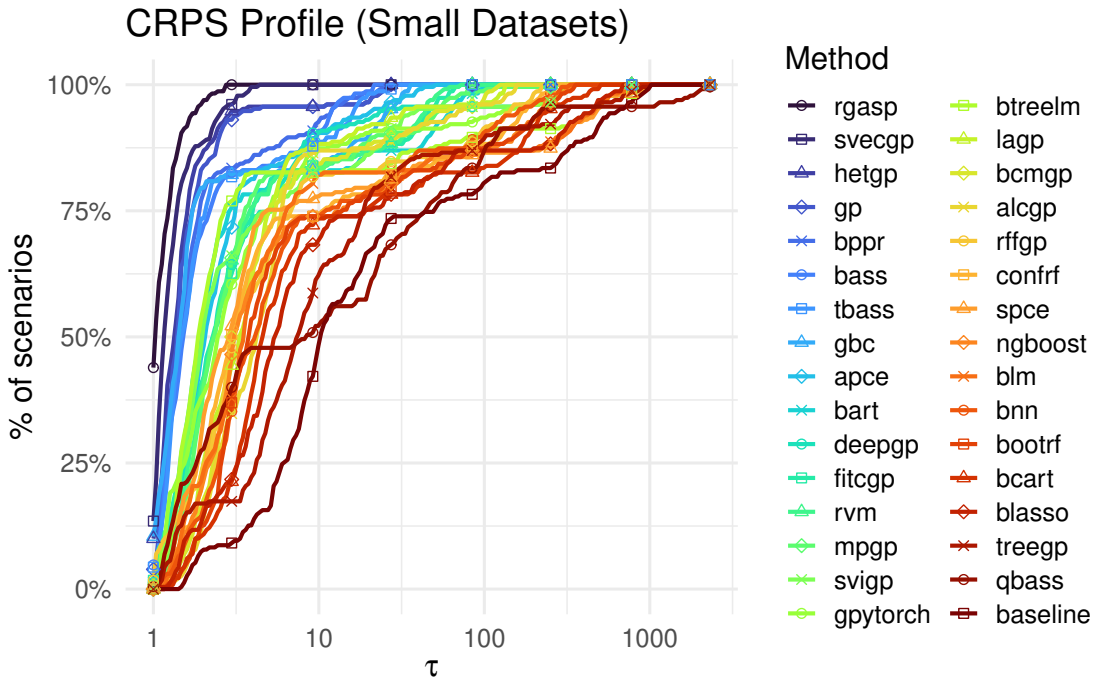


Figure 8: Performance profile plots for the real datasets with training size $n < 2000$. For each emulator, the curve shows the proportion of datasets for which the method is within a factor τ of the best-performing emulator in terms of CRPS. Curves closer to the upper-left corner indicate stronger and more consistent performance. Note that TREEGP only completed on eleven datasets in the allotted time and was replaced by the baseline model for the remaining cases.

5 Discussion and Analysis

5.1 There is no free lunch (or best emulator)

In uncertainty quantification, it is common to ask: *which emulator is the best?* But this is rarely the most productive question to pursue. The no free lunch theorems [Wolpert, 2002] guarantee that no emulator can possibly "win" on all functions. Indeed, of the 32 emulators in this study, 28 of them had the lowest CRPS value on at least one of the simulation scenarios. Six (out of many) notable cases are shown in Figure 13, demonstrating that different emulators can dominate for different functions. The DEEPGP emulator is an interesting case study: this emulator does reasonably well in the synthetic data simulation study with an overall win rate of 9.6% and an average ranking of $11.0/32$. But DEEPGP was not designed to be a catch-all emulator; it was designed to be effective for challenging nonstationary emulation problems where sophisticated tools are required and, in these settings, it does exceptionally well. On test functions such as $\mathrm{foursquare}$,

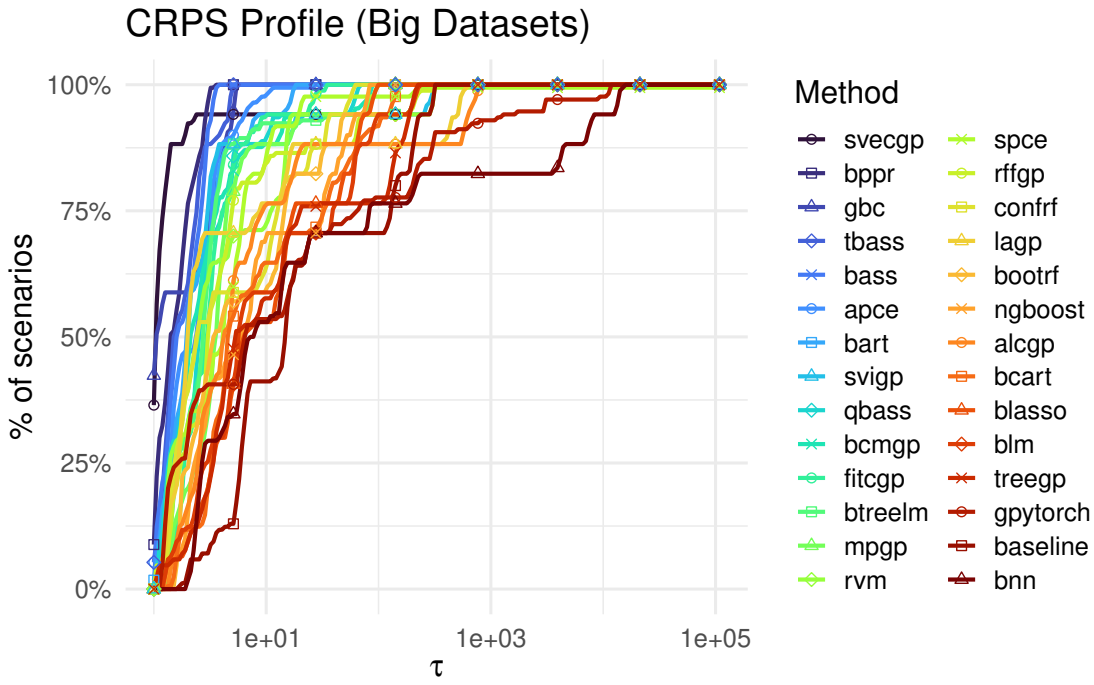
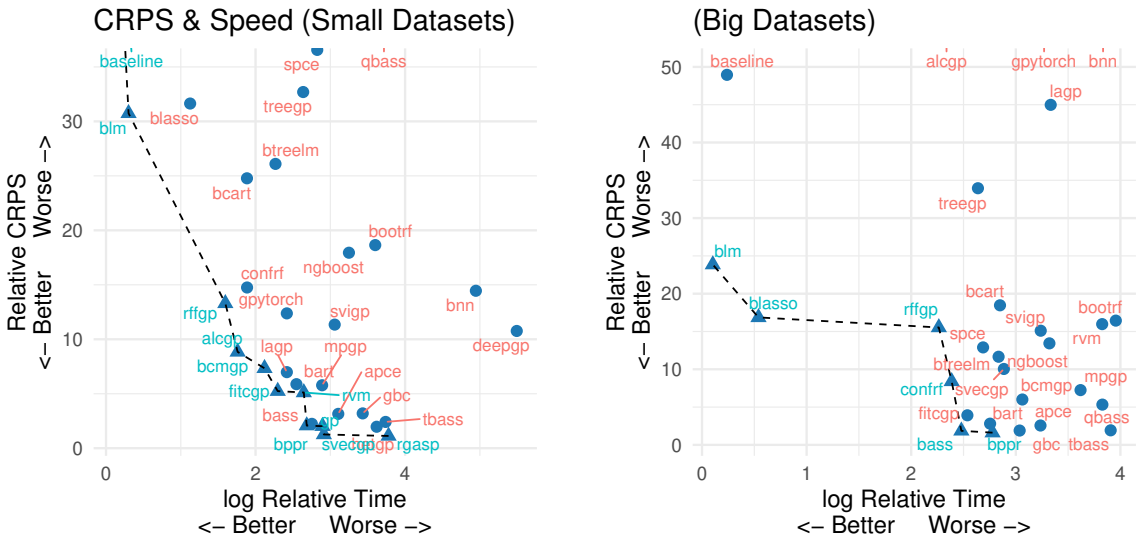


Figure 9: Performance profile plots for the real datasets with training size $n \geq 2000$. For each emulator, the curve shows the proportion of datasets for which the method is within a factor τ of the best-performing emulator in terms of CRPS. Curves closer to the upper-left corner indicate stronger and more consistent performance. Several slower emulators were excluded from this analysis.

squiggle, star2, and ignition, it often exhibits the best CRPS by a wide margin (see Figure 13a for an example).

For another example, consider two of the standout fast emulators in this study: SVECGP and BPPR. The SVECGP emulator had an overall win rate of 11.8% in the synthetic data simulations, with an average rank of 7.2 out of 32. By comparison, BPPR achieved the best CRPS in just 3.9% of scenarios and had an average rank of 7.7. However, the average CRPS relative to the best model (see Equation (2); $\omega = 0.001$) was 32.9 for SVECGP and only 10.0 for BPPR, suggesting that BPPR was often more consistently and practically competitive even when it did not win outright. Stratifying by noise-to-signal ratio reveals a modest shift in their relative behavior: SVECGP has a head-to-head win rate of 70.0% against BPPR when $NSR = 0$, but only 48.7% when $NSR = 0.1$. The corresponding average ranks tell a similar story: SVECGP averaged 7.0 and 7.6 in the $NSR = 0$ and $NSR = 0.1$ settings, compared with 8.1 and 7.1 for BPPR. Taken together, these results reinforce the no-free-lunch concept. Even among strong fast emulators, relative performance



(a) (b)

Figure 10: Pareto front of emulators based on average relative CRPS and log relative runtime across all simulation scenarios. Relative CRPS is computed as $(\text{CRPS} + \omega)/(\min \text{CRPS} + \omega)$ for each scenario, capped at 1000 to reduce sensitivity to outliers ($\omega = 0.001$). The dashed line and triangle markers indicate the methods which are not dominated (slower and less accurate) by any other emulator.

depends on the structure of the problem, with SVECGP favored more often in noise-free settings and BPPR becoming more competitive when noise is present.

These results suggest that searching for a single emulator that performs best across all functions is a fruitless endeavor. Moreover, searching for the best emulator in a *single* setting may also be misguided. "*Amazing things come from having many good models*" and "*all models are wrong but* **many** *are useful*" describe the sentiment behind the Rashomon effect, an idea that has gained increasing attention in recent years [Fisher et al., 2019, Xin et al., 2022, Rudin et al., 2024, Visokay et al., 2025]. The term 'Rashomon effect' was originally coined by Leo Breiman to describe the phenomenon in which multiple distinct models explain the data equally well, yet may lead to different conclusions [Breiman, 2001]. When this occurs (in sensitivity analysis or Bayesian model calibration, for example) it may signal underlying limitations in the modeling assumptions or data, including potential issues related to parameter identifiability or model discrepancy. Conversely, agreement among similarly accurate (yet distinct) emulators can increase confidence in the results. A short case study illustrating this idea is provided in the supplement.

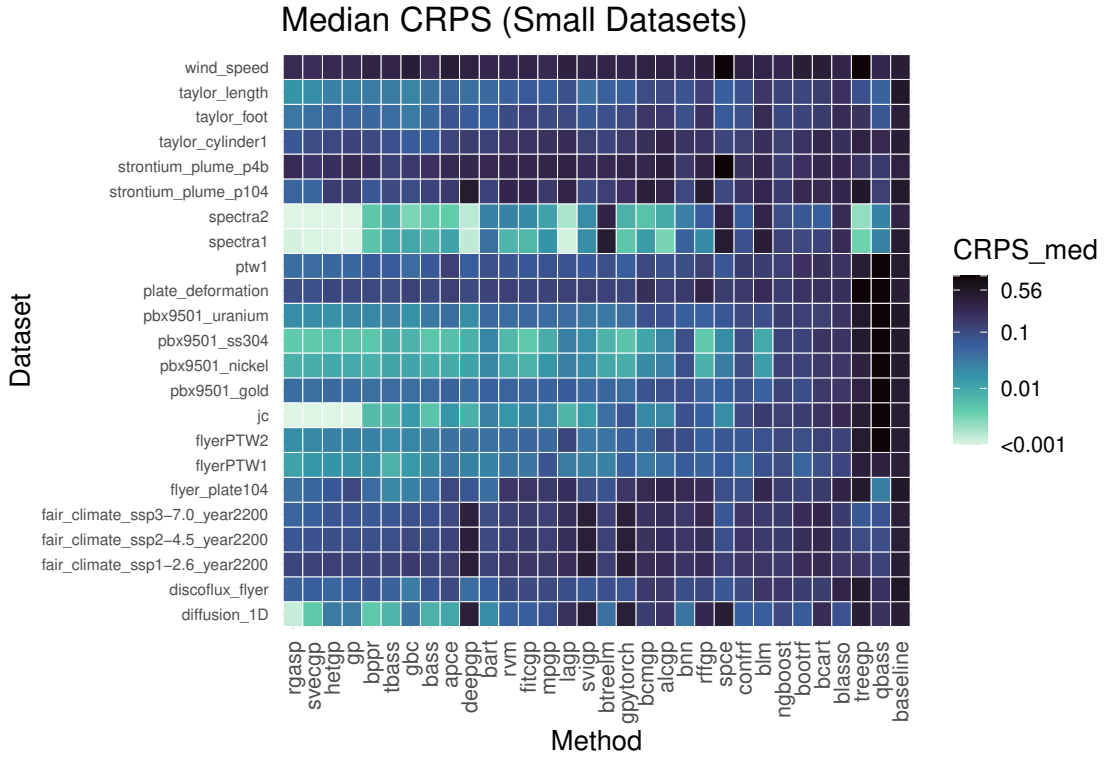


Figure 11: A heatmap of CRPS (truncated between 0.001 and 1.0 for visual simplicity) for the real datasets with $n < 2000$.

5.2 Tuning an emulator

To demonstrate how `duqling` can support exploration of tuning parameters, we re-ran the simulation study with 32 variants of the LAGP emulator, modifying three key parameters: the neighborhood size, the local design criterion, and the nugget size. Specifically, we considered neighborhood sizes of 25, 50, 100, and 200, and four selection criteria—`nn`, `alcray`, `alc`, and `mspe`—ranging from fastest to slowest in terms of computational cost (see Gramacy and Apley [2015] for details). We also compared two different nugget values: the default value of 0.0001 and a data-scaled nugget value (usually smaller, as suggested by the LAGP documentation) equal to the response variance divided by 10^7. These variants were treated as new emulators and *joined* with the existing simulation results (see Section 3).

Figure 14 shows the resulting Pareto front. The dynamic nugget choice does lead to a substantial improvement in performance (and computational efficiency) in this setting. Interestingly, while neighborhood size has a noticeable impact on runtime, its effect on CRPS performance is relatively small. The same is broadly true for

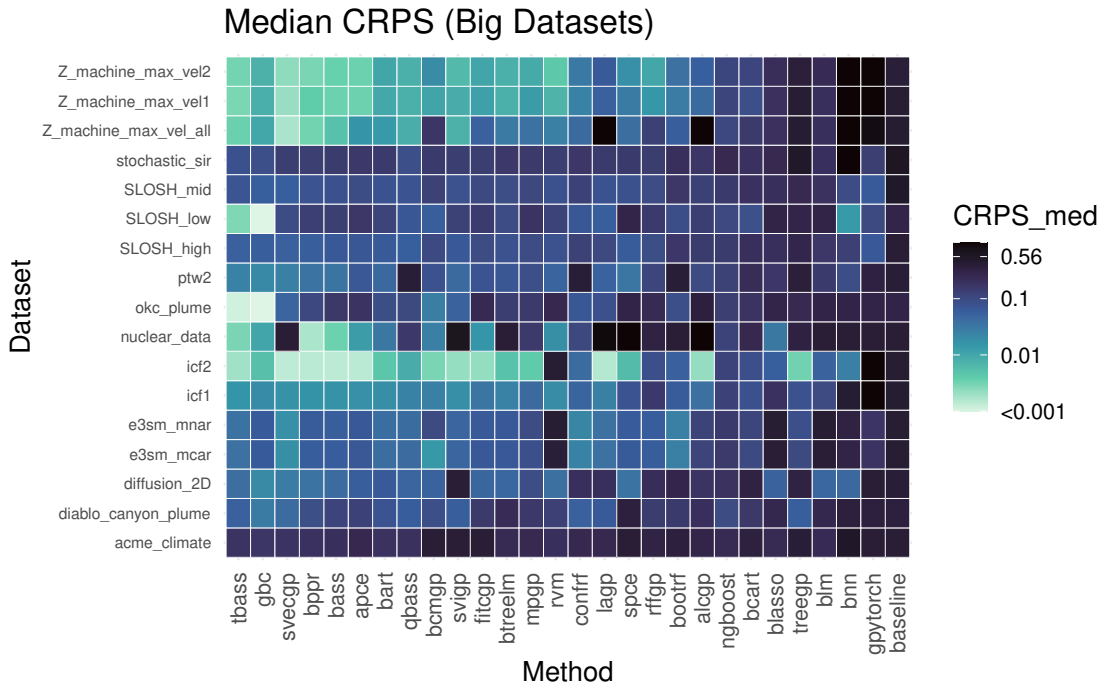


Figure 12: A heatmap of CRPS (truncated between 0.001 and 1.0 for visual simplicity) for the real datasets with $n \geq 2000$.

the selection criterion, though its impact on performance appears more pronounced when the neighborhood size is small. For this metric and set of test functions, a favorable tradeoff between speed and accuracy is achieved by choosing either (i) the more sophisticated `alc` criterion with a neighborhood size of 25, or (ii) the faster `nn` criterion with a neighborhood size of 100.

It is worth noting that the results in this section are somewhat sensitive to the choice of metric. For example, if the relative CRPS value is calculated with $\omega = 0$ (see Equation (2)), then the differences between methods become negligible, and the default nugget is even preferred in some cases. See `github.com/knrumsey/duqling_results/R/duqling/tuning.R` for details and for additional exploration of alternative metrics.

While this example focuses on LAGP, the same approach can be applied to any emulator. The ability to treat tuned variants as distinct methods makes it easy to identify effective default settings, or to study how emulator behavior responds to different hyperparameters. This can be a valuable tool both for practitioners selecting models and for researchers developing new emulation strategies.

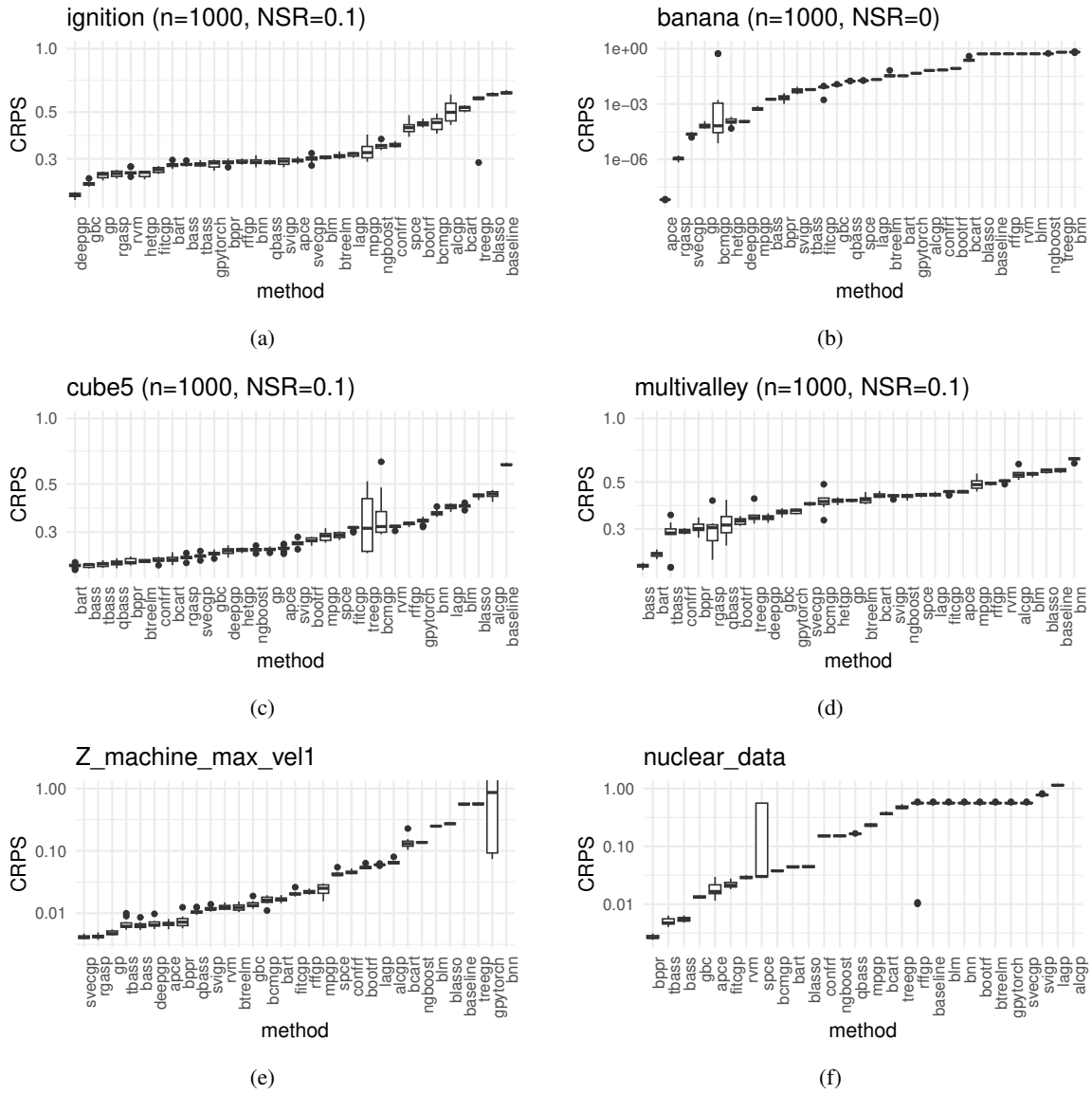


Figure 13: Boxplots of CRPS across ten replications/folds for four test functions and two real datasets. In each panel, a different emulator seems to be the "best" for that particular scenario.

5.3 Clustering by performance

We also explore the structure of these results through a simple clustering analysis. Rather than focusing on individual comparisons, we ask *which emulators behave similarly across scenarios, and which datasets elicit similar behavior from the emulators?* To accomplish this, we construct rank-based performance vectors for emulators (across all datasets and test functions) and for datasets (across all emulators), then compute

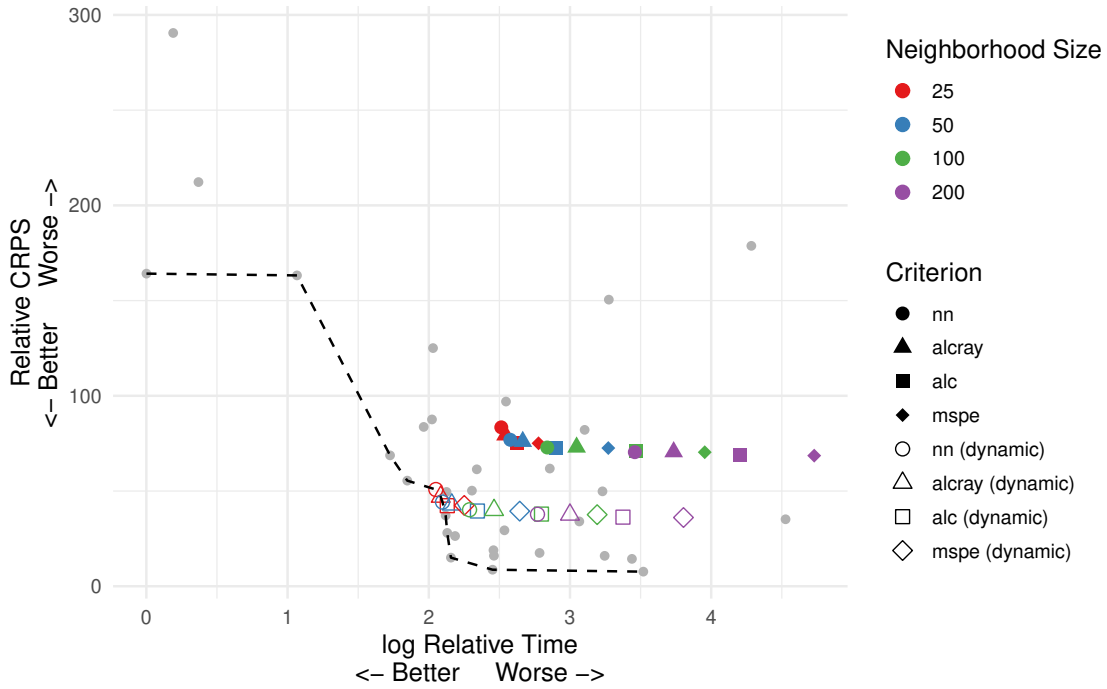
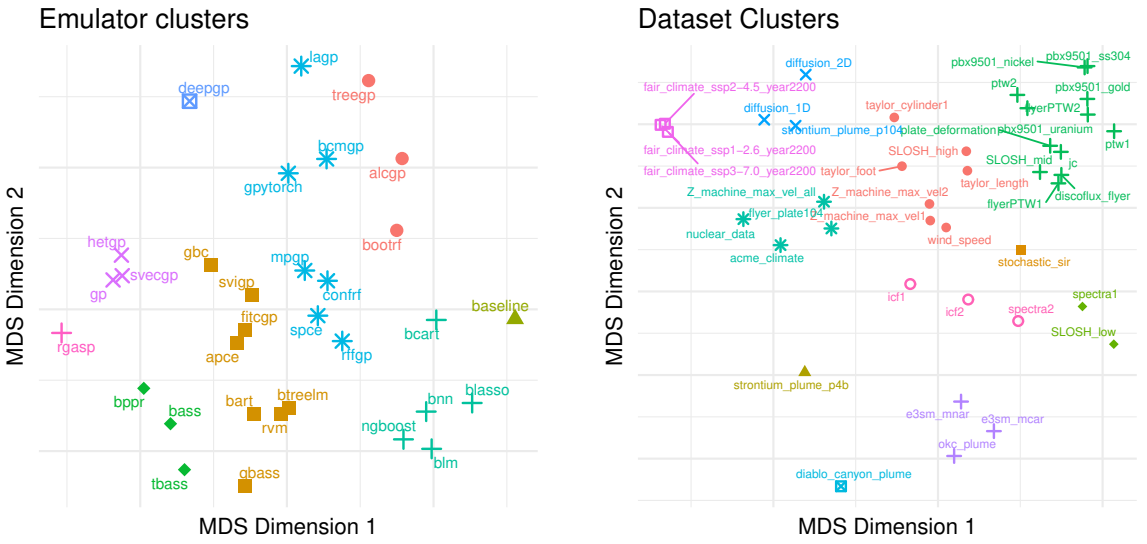


Figure 14: The Pareto front of Figure 5a is recreated here, with the addition of 32 new LAGP emulators with various hyperparameter values.

pairwise distances between these vectors. We visualize the resulting geometry using classical multidimensional scaling and identify clusters using DBSCAN. Figure 15 illustrates the results of this analysis using multidimensional scaling and DBSCAN clustering [Ester et al., 1996].

On the emulator side, distinct classes of methods emerge, and even within those families the relative distances carry information. The basis function approaches BPPR and BASS sit tightly together, with related methods such as RFFGP, APCE, and BART somewhat nearby. Tree-based methods form another distinct group in the bottom right. Several Gaussian process clusters appear: the SoD-adjacent methods ALCGP, MPGP, FITCGP, and BCMGP are all close together; DEEPGP and TREEGP show up as nearest neighbors; and the classical GP and its heteroskedastic counterpart HETGP naturally cluster. For a new dataset, Figure 15a may help a practitioner identify a sensible set of candidate emulators. While considering 32+ candidates may be unnecessary, it is often reasonable to include at least one representative from each major group.

On the dataset side, clustering reveals natural classes of problems and also points toward opportunities for future work. For any given emulator, it is useful to know which classes of datasets it tends to succeed on, and which expose its weaknesses. In some cases it may make sense to double down on problem types where



(a) A clustering of emulators using rank-based performance similarity and multidimensional scaling. Nearby emulators performed similarly on the various datasets and test functions.

(b) A clustering of datasets using rank-based performance similarity and multidimensional scaling. Nearby datasets led to similar performance by the various emulators.

Figure 15

a method already excels, while in others one might identify weak spots and build targeted extensions. For example, if one cluster of datasets is consistently challenging for an entire family of emulators, that cluster becomes a clear target for specialized method development (the strontium_plume_p4b dataset seems to be a ripe challenge). Another potential use is to match synthetic test functions to real datasets: functions that cluster closely with observed data may be especially valuable as benchmarks. For brevity, additional clustering plots, including one that combines both test functions and datasets, are provided in the supplement.

5.4 How to pick an emulator

If the practitioner takes just one thing away from this work, it should be that there are many choices and considerations when it comes to choosing an emulator. We strongly encourage examining a suite of candidates, at least 1 from each cluster in Figure 15a, before choosing one (or many, as suggested by the Rashomon effect).

For many users, predictive accuracy is the primary (if not the only) criterion for selecting an emulator. Some emulators perform exceptionally well on specific functions and possibly not so well on others. Some emulators perform well across a broad range of scenarios despite rarely being "the best". Depending on how

well one understands the underlying data, this tradeoff between robustness and peak performance may be an important consideration.

Of course, there are many reasons to choose an emulator that have less to do with raw accuracy. When the simple BLM works well enough, it brings with it at least a century of statistical insight. Everything from uncertainty quantification to hypothesis testing to experimental design has been deeply studied in that framework. Similarly, polynomial chaos expansions make global sensitivity analysis via Sobol indices essentially trivial [Sudret et al., 2017], though other emulators like BASS and BART can also provide closed-form estimates [Francom et al., 2018, Zamanian et al., 2021, Horiguchi and Pratola, 2023].

Some emulators are better suited to particular modeling goals. For example, Gaussian processes and basis regression models support closed-form active subspace directions [Constantine, 2015], which can help uncover low-dimensional structure [Wycoff et al., 2021, Rumsey et al., 2024b]. Other methods are designed around specific UQ tasks—such as expected improvement for Bayesian optimization [Jones et al., 1998], surrogate modeling across multiple simulators [Yannotty et al., 2024, Motamed, 2020], simulators with categorical inputs [Zhang et al., 2021, Francom et al., 2019a], or analyzing adjacent computer models [Bernstein et al., 2019, Rumsey et al., 2025]. A related trend is the development of physics-informed surrogates, which can outperform purely data-driven models by incorporating structural knowledge [McClarren et al., 2011, Kashinath et al., 2021, Dalton et al., 2023, Orjuela-Quintana et al., 2026]. Sometimes specialized emulators are required to accommodate particular data structures [Da Veiga and Marrel, 2012, Mallasto and Feragen, 2018, Flowers et al., 2026]. In other cases, emulators are popular because they are familiar and trusted by a community: RVM extends support vector machines, and CONFRF builds on random forests using conformal inference for fast model-free UQ. Stochastic simulators often require specialized treatment, and much research has focused on this challenge [Baker et al., 2020, Rumsey et al., 2024a, Binois and Gramacy, 2021, Plumlee and Tuo, 2014, Oakley and Youngman, 2017, Zhu and Sudret, 2023, Lüthen et al., 2023]. Functional or vector-valued outputs (e.g., spatial or temporal fields) present additional challenges for emulation, and often require specialized modeling strategies or extensions of standard surrogate methods [Liu and West, 2004, Higdon et al., 2008, Conti and O'Hagan, 2010, Gu and Berger, 2016, Francom et al., 2025]. Many important applications fall into this class, and extending the framework to these settings is an important direction for future work in `duqling`. Emulation with high-dimensional inputs (e.g., $p > n$) may also require customization, though approaches that perform a model search like tree-based or basis function models can work well without modification [Chakraborty et al., 2017, Lataniotis et al., 2020, Rumsey et al., 2024b, Shustin et al., 2026]. Hybrid approaches, which combine elements of various emulator types, are also a com-

mon approach for exploiting strengths and mitigating weaknesses. For example, Gaussian processes have been combined with spline representations [Chakraborty et al., 2013, Bowman and Woods, 2016], polynomial chaos expansions [Schobi et al., 2015, Zhou et al., 2019], and Bayesian regression trees [Maia et al., 2024].

Ultimately, the right emulator may depend critically on the task it is intended for. If training is a one-time cost and downstream inference is fast, a more complex, state-of-the-art model may be justified [Radev et al., 2023]. If hundreds of emulators are required, a training-free method like LAGP can be especially appealing [Gramacy and Apley, 2015]. In settings requiring millions of sequential predictions, the choice of emulator should reflect that constraint [Rumsey et al., 2023, Braun et al., 2024, Hutchings et al., 2024]. In this sense, downstream context should shape your emulator selection just as much (if not more) than eking out that last percent of CRPS performance.

6 Conclusion

This paper presents a large-scale, reproducible comparison of 32 emulators across 60 synthetic test functions and 40 real-world datasets. While no single emulator dominates in all settings, the results highlight important tradeoffs between accuracy, robustness, and runtime. Notably, GP [Binois and Gramacy, 2021] and RGASP [Gu et al., 2018] both demonstrate strong predictive performance, with some evidence of a time–accuracy tradeoff between them. The SVECGP emulator stands out for its excellent scaling and reliable accuracy across a wide range of scenarios [Katzfuss et al., 2022]. We also find that BPPR performs consistently well across a broad range of settings, with particularly strong performance in the presence of noise [Collins et al., 2024].

The `duqling` framework, available at `github.com/knrumsey/duqling` and `github.com/reidmorris/duqling_py`, played a central role in enabling this study and is designed to support future emulator benchmarking, tuning, and development. All results in this paper were generated using open infrastructure that can be extended or modified to suit new problems, helping to make emulator evaluation both rigorous and reproducible.

A natural next step is to extend this framework to a broader class of emulation problems. While this study focuses on scalar-output, deterministic simulators with continuous inputs, many important applications involve additional complexities. In particular, future work will involve extending this framework to conduct automated, large-scale, and reproducible comparisons in settings with categorical inputs, stochastic simulators, and multivariate or functional outputs. These settings introduce new modeling challenges and tradeoffs,

and we expect that some of the lessons from this study will carry over, while also highlighting differences in emulator behavior in these more complex settings.

We acknowledge that several of the emulators evaluated in this paper, including BASS, BPPR, and APCE, were developed in part by the authors of this study. We have tried to be transparent and fair in our analysis, with the goal of supporting reproducible comparisons. Our motivation is to move the field forward, not to push our own methods, and we are excited by the success (and the potential for even greater future success) of emulators developed by others in the UQ community. We encourage the interested reader to compare their own results to those in this paper using `duqling`.

Supplemental Materials

Additional details, figures, and implementation information are provided in the supplement. Section SM1 contains the R scripts used to reproduce all results. Section SM2 summarizes emulator performance across all simulation scenarios, including overall summaries and breakdowns by test function and dataset. Section SM3 provides technical details on the reproducible simulation study framework, including seed construction, fallback models, output formats, and descriptions of the datasets and test functions. Section SM4 presents additional discussion of the Rashomon effect and ensembling. Section SM5 includes extended figures and analyses for multiple performance metrics (e.g., FVU, CRPS, and runtime), along with additional diagnostics such as performance profiles, heatmaps, and boxplots.

Acknowledgements

We would like to thank the editors, associate editor, and the two anonymous reviewers for their feedback, which has led to a much improved paper. We are also grateful to Derek Bingham, Gavin Collins, Annie Booth, Matthias Katzfuss, Bruno Sudret, Bobby Gramacy, Mengyang Gu, and C. A. Eiffer for valuable discussions and, in some cases, feedback on earlier versions of this manuscript.

References

R. Alizadeh, J. K. Allen, and F. Mistree. Managing computational complexity using surrogate models: a critical review. *Research in Engineering Design*, 31(3):275–298, 2020.

E. Baker, P. Challenor, and M. Eames. Predicting the output from a stochastic computer model when a deterministic approximation is available. *Journal of Computational and Graphical Statistics*, 29(4):786–797, 2020.

E. Baker, P. Barbillon, A. Fadikar, R. B. Gramacy, R. Herbei, D. Higdon, J. Huang, L. R. Johnson, P. Ma, A. Mondal, et al. Analyzing stochastic computer models. *Statistical Science*, 37(1):64–89, 2022.

J. Bernstein, K. Schmidt, D. Rivera, N. Barton, J. Florando, and A. Kupresanin. A comparison of material flow strength models using Bayesian cross-validation. *Computational Materials Science*, 169:109098, 2019.

M. Binois and R. B. Gramacy. hetgp: Heteroskedastic Gaussian process modeling and sequential design in R. *Journal of Statistical Software*, 98:1–44, 2021.

M. Binois, R. B. Gramacy, and M. Ludkovski. Practical heteroscedastic Gaussian process modeling for large simulation experiments. *Journal of Computational and Graphical Statistics*, 27(4):808–821, 2018.

A. Biswas, D. C. Francom, J. N. Plohr, S. K. Sjue, P. R. Trubey, D. J. Walters, and D. J. Luscher. ASC/PEM/-Materials L3 Milestone Briefing: A briefing on Impala and the Savanna efforts in support of FY22/FY23 SCDS L1 milestones. Technical report, Los Alamos National Laboratory (LANL), Los Alamos, NM (United States), 2021.

V. E. Bowman and D. C. Woods. Emulation of multivariate simulators using thin-plate splines with application to atmospheric dispersion. *SIAM/ASA Journal on Uncertainty Quantification*, 4(1):1323–1344, 2016.

T. Braun, A. Kvellestad, and R. De Bin. GPTreeO: An R package for continual regression with dividing local Gaussian processes. *arXiv preprint arXiv:2410.01024*, 2024.

L. Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

L. Breiman, J. Friedman, R. A. Olshen, and C. J. Stone. *Classification and regression trees*. Chapman and Hall/CRC, 2017.

J. L. Brown and L. B. Hund. Estimating material properties under extreme conditions by using bayesian model calibration with functional outputs. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 67(4):1023–1045, 2018.

M. J. Brown, A. Gowardhan, M. Nelson, M. Williams, and E. R. Pardyjak. Evaluation of the quic wind and dispersion models using the joint urban 2003 field experiment dataset. In *AMS 8th Symp. Urban Env, Phoenix, USA*, pages 10–16, 2009.

B. Carpenter, A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li, and A. Riddell. Stan: A probabilistic programming language. *Journal of statistical software*, 76:1–32, 2017.

A. Chakraborty, B. K. Mallick, R. G. Mcclarren, C. C. Kuranz, D. Bingham, M. J. Grosskopf, E. M. Rutter, H. F. Stripling, and R. P. Drake. Spline-based emulators for radiative shock experiments with measurement error. *Journal of the American Statistical Association*, 108(502):411–428, 2013.

A. Chakraborty, D. Bingham, S. S. Dhavala, C. C. Kuranz, R. P. Drake, M. J. Grosskopf, E. M. Rutter, B. R. Torralva, J. P. Holloway, R. G. McClarren, et al. Emulation of numerical models with over-specified basis functions. *Technometrics*, 59(2):153–164, 2017.

S. Chatterjee. *bnns: Bayesian Neural Network with 'Stan'*, 2025. URL `https://CRAN.R-project.org/package=bnns`. R package version 0.1.2.

T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.

H. A. Chipman, E. I. George, and R. E. McCulloch. Bayesian cart model search. *Journal of the American Statistical Association*, 93(443):935–948, 1998.

H. A. Chipman, E. I. George, and R. E. McCulloch. Bayesian treed models. *Machine Learning*, 48(1): 299–320, 2002.

H. A. Chipman, E. I. George, and R. E. McCulloch. BART: Bayesian additive regression trees. *Annals of Applied Statistics*, 6(1):266–298, 2012.

G. Collins, D. Francom, and K. N. Rumsey. Bayesian projection pursuit regression. *Statistics and Computing*, 34(1):29, 2024.

P. G. Constantine. *Active subspaces: Emerging ideas for dimension reduction in parameter studies*. SIAM, 2015.

S. Conti and A. O'Hagan. Bayesian emulation of complex multi-output and dynamic computer models. *Journal of statistical planning and inference*, 140(3):640–651, 2010.

C. Cortes and V. Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.

A. I. Cowen-Rivers, W. Lyu, R. Tutunov, Z. Wang, A. Grosnit, R. R. Griffiths, A. M. Maraval, H. Jianye, J. Wang, J. Peters, et al. Hebo: Pushing the limits of sample-efficient hyper-parameter optimisation. *Journal of Artificial Intelligence Research*, 74:1269–1349, 2022.

N. Cressie. The origins of kriging. *Mathematical geology*, 22(3):239–252, 1990.

S. Da Veiga and A. Marrel. Gaussian process modeling with inequality constraints. In *Annales de la Faculté des sciences de Toulouse: Mathématiques*, volume 21, pages 529–555, 2012.

W. Dabney, G. Ostrovski, D. Silver, and R. Munos. Implicit quantile networks for distributional reinforcement learning. In *International conference on machine learning*, pages 1096–1105. PMLR, 2018.

D. Dalton, D. Husmeier, and H. Gao. Physics-informed graph neural network emulation of soft-tissue mechanics. *Computer Methods in Applied Mechanics and Engineering*, 417:116351, 2023.

A. Damianou and N. D. Lawrence. Deep Gaussian processes. In *Artificial intelligence and statistics*, pages 207–215. PMLR, 2013.

M. Deisenroth and J. W. Ng. Distributed Gaussian processes. In *International conference on machine learning*, pages 1481–1490. PMLR, 2015.

D. G. Denison, B. K. Mallick, and A. F. Smith. Bayesian MARS. *Statistics and Computing*, 8(4):337–346, 1998a.

D. G. Denison, B. K. Mallick, and A. F. Smith. A bayesian cart algorithm. *Biometrika*, 85(2):363–377, 1998b.

D. G. Denison, C. C. Holmes, B. K. Mallick, and A. F. Smith. *Bayesian methods for nonlinear classification and regression*. John Wiley & Sons, 2002.

E. D. Dolan and J. J. Moré. Benchmarking optimization software with performance profiles. *Mathematical programming*, 91(2):201–213, 2002.

T. Duan, A. Anand, D. Y. Ding, K. K. Thai, S. Basu, A. Ng, and A. Schuler. Ngboost: Natural gradient boosting for probabilistic prediction. In *International conference on machine learning*, pages 2690–2700. PMLR, 2020.

T. Edmunds, A. Lamont, V. Bulaevskaya, C. Meyers, J. Mirocha, A. Schmidt, M. Simpson, S. Smith, P. Sotorrio, P. Top, et al. The value of storage and demand response for renewble integration. Technical report, Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States), 2013.

B. Efron. The bootstrap and modern statistics. *Journal of the American Statistical Association*, 95(452): 1293–1296, 2000.

M. Ester, H.-P. Kriegel, J. Sander, X. Xu, et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, volume 96, pages 226–231, 1996.

M. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim. Do we need hundreds of classifiers to solve real world classification problems? *The journal of machine learning research*, 15(1):3133–3181, 2014.

A. Fisher, C. Rudin, and F. Dominici. All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20(177):1–81, 2019.

A. R. Flowers, C. T. Franck, M. Binois, C. Park, and R. B. Gramacy. Modular jump Gaussian processes. *Data Science in Science*, 5(1):2612651, 2026.

D. Francom and B. Sansó. BASS: An R package for fitting and performing sensitivity analysis of bayesian adaptive spline surfaces. *Journal of Statistical Software*, 94(1):1–36, 2020.

D. Francom, B. Sansó, A. Kupresanin, and G. Johannesson. Sensitivity analysis and emulation for functional data using bayesian adaptive splines. *Statistica Sinica*, pages 791–816, 2018.

D. Francom, B. Sansó, V. Bulaevskaya, D. Lucas, and M. Simpson. Inferring atmospheric release characteristics in a large computer experiment using bayesian adaptive splines. *Journal of the American Statistical Association*, 114(528):1450–1465, 2019a.

D. Francom, D. J. Walters, J. L. Barber, D. J. Luscher, E. Lawrence, A. Biswas, C. M. Biwer, D. Banesh, J. Lazarz, S. C. Vogel, et al. Simulation and emulation of x-ray diffraction from dynamic compression experiments. *Journal of Dynamic Behavior of Materials*, 7(2):170–187, 2021.

D. Francom, J. D. Tucker, G. Huerta, K. Shuler, and D. Ries. Elastic bayesian model calibration. *SIAM/ASA Journal on Uncertainty Quantification*, 13(1):195–227, 2025.

D. C. Francom, S. A. Vander Wiel, and B. P. Weaver. Nuclear data dimension reduction. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2019b.

J. H. Friedman. Multivariate adaptive regression splines. *The annals of statistics*, pages 1–67, 1991.

J. H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001.

J. H. Friedman and W. Stuetzle. Projection pursuit regression. *Journal of the American statistical Association*, 76(376):817–823, 1981.

J. A. Gaffney, R. Anirudh, P.-T. Bremer, J. Hammer, D. Hysom, S. A. Jacobs, J. L. Peterson, P. Robinson, B. K. Spears, P. T. Springer, J. J. Thiagarajan, B. Van Essen, and J.-S. Yeom. The jag inertial confinement fusion simulation dataset for multi-modal scientific deep learning. In Lawrence Livermore Na-

tional Laboratory (LLNL) Open Data Initiative. UC San Diego Library Digital Collections., 3 2020. URL https://library.ucsd.edu/dc/object/bb5534097t.

J. Gardner, G. Pleiss, K. Q. Weinberger, D. Bindel, and A. G. Wilson. Gpytorch: Blackbox matrix-matrix Gaussian process inference with gpu acceleration. *Advances in neural information processing systems*, 31, 2018.

L. Gautier. rpy2: A python interface to R. https://rpy2.github.io/, 2026. Python package.

A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin. *Bayesian Data Analysis*. CRC Press, 2013.

A. Gelman et al. Prior distributions for variance parameters in hierarchical models (comment on article by browne and draper). *Bayesian analysis*, 1(3):515–534, 2006.

T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American statistical Association*, 102(477):359–378, 2007.

R. B. Gramacy. laGP: large-scale spatial modeling via local approximate Gaussian processes in R. *Journal of Statistical Software*, 72:1–46, 2016.

R. B. Gramacy. *Surrogates: Gaussian process modeling, design, and optimization for the applied sciences*. CRC press, 2020.

R. B. Gramacy and D. W. Apley. Local Gaussian process approximation for large computer experiments. *Journal of Computational and Graphical Statistics*, 24(2):561–578, 2015.

R. B. Gramacy and H. K. H. Lee. Bayesian treed Gaussian process models with an application to computer modeling. *Journal of the American Statistical Association*, 103(483):1119–1130, 2008.

P. J. Green. Reversible jump Markov chain Monte Carlo computation and Bayesian model determination. *Biometrika*, 82(4):711–732, 1995.

M. Grosskopf, E. Lawrence, A. Biswas, L. Tang, K. Rumsey, L. Van Roekel, and N. Urban. In-situ spatial inference on climate simulations with sparse Gaussian processes. In *ISAV'21: In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*, pages 31–36. 2021.

M. Gu and J. O. Berger. Parallel partial Gaussian process emulation for computer models with massive output. *The Annals of Applied Statistics*, pages 1317–1347, 2016.

M. Gu, X. Wang, and J. O. Berger. Robust Gaussian stochastic process emulation. *The Annals of Statistics*, 46(6A):3038–3066, 2018.

M. Gu, J. Palomo, and J. O. Berger. RobustGaSP: Robust Gaussian stochastic process emulation in R. *The R Journal*, 11:112–136, 2019. ISSN 2073-4859. doi: 10.32614/RJ-2019-011. https://doi.org/10.32614/RJ-2019-011.

J. Guinness, M. Katzfuss, and Y. Fahmy. *GpGp: Fast Gaussian Process Computation Using Vecchia's Approximation*, 2024. URL `https://CRAN.R-project.org/package=GpGp`. R package version 0.5.1.

M. J. Heaton, A. Datta, A. O. Finley, R. Furrer, J. Guinness, R. Guhaniyogi, F. Gerber, R. B. Gramacy, D. Hammerling, M. Katzfuss, et al. A case study competition among methods for analyzing large spatial data. *Journal of agricultural, biological and environmental Statistics*, 24(3):398–425, 2019.

J. Hensman, N. Fusi, and N. D. Lawrence. Gaussian processes for big data. In *29th Conference on Uncertainty in Artificial Intelligence, UAI 2013*, pages 282–290, 2013.

D. Higdon, J. Gattiker, B. Williams, and M. Rightley. Computer model calibration using high-dimensional output. *Journal of the American Statistical Association*, 103(482):570–583, 2008.

A. Hlobilová, S. Marelli, and B. Sudret. Surrogate modeling benchmark - two-dimensional heat diffusion model, Sept. 2024a. URL `https://doi.org/10.5281/zenodo.12701147`.

A. Hlobilová, S. Marelli, and B. Sudret. Surrogate modeling benchmark - one-dimensional diffusion model, Sept. 2024b. URL `https://doi.org/10.5281/zenodo.12704504`.

A. Horiguchi and M. T. Pratola. Estimating shapley effects in big-data emulation and regression settings using bayesian additive regression trees. *arXiv preprint arXiv:2304.03809*, 2023.

G. Hutchings, B. Sansó, J. Gattiker, D. Francom, and D. Pasqualini. Comparing emulation methods for a high-resolution storm surge model. *Environmetrics*, 34(3):e2796, 2023.

G. Hutchings, D. Bingham, K. N. Rumsey, and E. Lawrence. Fast emulation, modular calibration, and active learning for simulators with functional response. *arXiv preprint arXiv:2405.16298*, 2024.

H. Ichimura. *Estimation of single index models*. PhD thesis, Massachusetts Institute of Technology, 1987.

IMPALA Team. `https://github.com/lanl/impala`, 2025.

U. Johansson, H. Boström, T. Löfström, and H. Linusson. Regression conformal prediction with random forests. *Machine learning*, 97(1):155–176, 2014.

D. R. Jones, M. Schonlau, and W. J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global optimization*, 13(4):455–492, 1998.

K. Kashinath, M. Mustafa, A. Albert, J. Wu, C. Jiang, S. Esmaeilzadeh, K. Azizzadenesheli, R. Wang, A. Chattopadhyay, A. Singh, et al. Physics-informed machine learning: case studies for weather and climate modelling. *Philosophical Transactions of the Royal Society A*, 379(2194):20200093, 2021.

M. Katzfuss and J. Guinness. A general framework for Vecchia approximations of Gaussian processes. *Statistical Science*, 36(1):124–141, 2021.

M. Katzfuss, J. Guinness, W. Gong, and D. Zilber. Vecchia approximations of Gaussian-Process predictions. *Journal of Agricultural, Biological and Environmental Statistics*, 25(3):383–414, 2020.

M. Katzfuss, J. Guinness, and E. Lawrence. Scaled vecchia approximation for fast computer-model emulation. *SIAM/ASA Journal on Uncertainty Quantification*, 10(2):537–554, 2022.

M. Katzfuss, M. Jurek, D. Zilber, and W. Gong. *GPvecchia: Scalable Gaussian-Process Approximations*, 2024. URL `https://CRAN.R-project.org/package=GPvecchia`. R package version 0.1.7.

S. Keerthi and W. Chu. A matching pursuit approach to sparse Gaussian process regression. *Advances in neural information processing systems*, 18, 2005.

M. C. Kennedy and A. O'Hagan. Bayesian calibration of computer models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 63(3):425–464, 2001.

A. Kigerl, Z. Hamilton, M. Kowalski, and X. Mei. The great methods bake-off: Comparing performance of machine learning algorithms. *Journal of Criminal Justice*, 82:101946, 2022.

N. Klein, J. Colgan, E. Judge, K. Bhat, K. Myers, and E. Lawrence. Atomic simulations and experimental data for basalt-like compounds, Oct. 2020. URL `https://doi.org/10.25583/1670467`.

E. Laloy and D. Jacques. Emulation of CPU-demanding reactive transport models: a comparison of Gaussian processes, polynomial chaos expansion, and deep neural networks. *Computational Geosciences*, 23(5): 1193–1215, 2019.

C. Lataniotis, S. Marelli, and B. Sudret. Extending classical surrogate modeling to high dimensions through supervised dimensionality reduction: a data-driven approach. *International Journal for Uncertainty Quantification*, 10(1), 2020.

D. V. Lindley and A. F. Smith. Bayes estimates for the linear model. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 34(1):1–18, 1972.

F. Liu and M. West. A dynamic modelling strategy for bayesian computer model emulation. *Bayesian Analysis*, 1(1), 2004.

H. Liu, Y.-S. Ong, X. Shen, and J. Cai. When Gaussian process meets big data: A review of scalable GPs. *IEEE transactions on neural networks and learning systems*, 31(11):4405–4423, 2020.

L. Lu, Y. Lu, S. Wang, K. Li, and X. Song. Survey and prospect of surrogate model technique and application. *Journal of Mechanical Engineering*, 60(3):254–281, 2024.

N. Lüthen, S. Marelli, and B. Sudret. A spectral surrogate model for stochastic simulators computed from trajectory samples. *Computer Methods in Applied Mechanics and Engineering*, 406:115875, 2023.

M. Maia, K. Murphy, and A. C. Parnell. GP-BART: A novel bayesian additive regression trees approach using Gaussian processes. *Computational Statistics & Data Analysis*, 190:107858, 2024.

A. Mallasto and A. Feragen. Wrapped Gaussian process regression on riemannian manifolds. In *Proceedings of the ieee conference on computer vision and pattern recognition*, pages 5580–5588, 2018.

F. W. Marrs and D. Francom. Multiclass classification using bayesian multivariate adaptive regression splines. *Bayesian Analysis*, 1(1):1–29, 2025.

R. G. McClarren, D. Ryu, R. P. Drake, M. Grosskopf, D. Bingham, C.-C. Chou, B. Fryxell, B. Van der Holst, J. P. Holloway, C. C. Kuranz, et al. A physics informed emulator for laser-driven radiating shock simulations. *Reliability Engineering & System Safety*, 96(9):1194–1207, 2011.

M. McKay, R. Beckman, and W. Conover. Comparison the three methods for selecting values of input variable in the analysis of output from a computer code. *Technometrics;(United States)*, 21(2), 1979.

J. J. Moré and S. M. Wild. Benchmarking derivative-free optimization algorithms. *SIAM Journal on Optimization*, 20(1):172–191, 2009.

M. Motamed. A multi-fidelity neural network surrogate sampling method for uncertainty quantification. *International Journal for Uncertainty Quantification*, 10(4), 2020.

R. M. Neal. *Bayesian learning for neural networks*, volume 118. Springer Science & Business Media, 2012.

D. J. Nott, A. Y. Kuk, and H. Duc. Efficient sampling schemes for bayesian mars models with many predictors. *Statistics and Computing*, 15(2):93–101, 2005.

J. E. Oakley and B. D. Youngman. Calibration of stochastic computer simulators using likelihood emulation. *Technometrics*, 59(1):80–92, 2017.

J. B. Orjuela-Quintana, D. Sapone, and S. Nesseris. Interpretable and physics-informed emulator for the linear matter power spectrum from machine learning. *Physical Review D*, 113(6):063526, 2026.

J. Ormerod, M. J. Davoudabadi, G. Tarr, S. Mueller, and J. Tidswell. *Bayesian Lasso Regression and Tools for the Lasso Distribution*, 2025. URL `https://garthtarr.github.io/BayesianLasso/`. R package version 0.3.0.

T. Park and G. Casella. The bayesian lasso. *Journal of the american statistical association*, 103(482):681–686, 2008.

M. Plumlee and R. Tuo. Building accurate emulators for stochastic simulations via quantile kriging. *Technometrics*, 56(4):466–473, 2014.

N. Polson and V. Sokolov. Generative bayesian computation as a scalable alternative to Gaussian process surrogates. *arXiv preprint arXiv:2602.21408*, 2026.

S. T. Radev, M. Schmitt, V. Pratz, U. Picchini, U. Köthe, and P.-C. Bürkner. Jana: Jointly amortized neural approximation of complex bayesian models. In *Uncertainty in Artificial Intelligence*, pages 1695–1706. PMLR, 2023.

A. Rahimi and B. Recht. Random features for large-scale kernel machines. *Advances in neural information processing systems*, 20, 2007.

C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.

S. Razavi, B. A. Tolson, and D. H. Burn. Review of surrogate modeling in water resources. *Water Resources Research*, 48(7), 2012.

C. Rudin, C. Zhong, L. Semenova, M. Seltzer, R. Parr, J. Liu, S. Katta, J. Donnelly, H. Chen, and Z. Boner. Position: Amazing things come from having many good models. *Proceedings of Machine Learning Research*, 235:42783–42795, 2024.

K. N. Rumsey, G. Huerta, and J. D. Tucker. A localized ensemble of approximate Gaussian processes for fast sequential emulation. *Stat*, 12(1):e576, 2023.

K. N. Rumsey, D. Francom, and A. Shen. Generalized Bayesian MARS: Tools for stochastic computer model emulation. *SIAM/ASA Journal on Uncertainty Quantification*, 12(2):646–666, 2024a.

K. N. Rumsey, D. Francom, and S. Vander Wiel. Discovering active subspaces for high-dimensional computer models. *Journal of Computational and Graphical Statistics*, 33(3):896–908, 2024b.

K. N. Rumsey, Z. K. Hardy, C. Ahrens, and S. Vander Wiel. Co-active subspace methods for the joint analysis of adjacent computer models. *Technometrics*, 67(1):133–146, 2025.

K. N. Rumsey, D. Francom, G. Gibson, J. Derek Tucker, and G. Huerta. Bayesian adaptive polynomial chaos expansions. *Stat*, 15(1):e70151, 2026.

J. M. Salter and D. Williamson. A comparison of statistical emulation methodologies for multi-wave calibration of environmental models. *Environmetrics*, 27(8):507–523, 2016.

K. Sargsyan, C. Safta, H. N. Najm, B. Debusschere, D. Ricciuto, and P. Thornton. Sparse Polynomial Chaos Surrogate for ACME Land Model via Iterative Bayesian Compressive Sensing. Technical report, Sandia National Lab.(SNL-CA), Livermore, CA (United States), 2015.

A. Sauer, A. Cooper, and R. B. Gramacy. Non-stationary Gaussian process surrogates. *arXiv preprint arXiv:2305.19242*, 2023a.

A. Sauer, A. Cooper, and R. B. Gramacy. Vecchia-approximated deep Gaussian processes for computer experiments. *Journal of Computational and Graphical Statistics*, 32(3):824–837, 2023b.

A. Sauer, R. B. Gramacy, and D. Higdon. Active learning for deep Gaussian process surrogates. *Technometrics*, 65(1):4–18, 2023c.

R. Schobi, B. Sudret, and J. Wiart. Polynomial-chaos-based kriging. *International Journal for Uncertainty Quantification*, 5(2), 2015.

Q. Shao, A. Younes, M. Fahs, and T. A. Mara. Bayesian sparse polynomial chaos expansion for global sensitivity analysis. *Computer Methods in Applied Mechanics and Engineering*, 318:474–496, 2017.

P. F. Shustin, S. Ubaru, M. J. Zimoń, S. Lu, V. Kalantzis, L. Horesh, and H. Avron. Pce-net: High-dimensional surrogate modeling for learning uncertainty. *SIAM/ASA Journal on Uncertainty Quantification*, 14(1):168–196, 2026.

S. Sjue, K. Hickmann, and N. Nguyen. Simple and sophisticated models of Taylor's cylinder impact. In *AIP Conference Proceedings*, volume 2844, page 370006. AIP Publishing LLC, 2023.

C. J. Smith, P. M. Forster, M. Allen, N. Leach, R. J. Millar, G. A. Passerello, and L. A. Regayre. FAIR v1. 3: a simple emissions-based impulse response and carbon cycle model. *Geoscientific Model Development*, 11(6):2273–2297, 2018.

A. Smola and P. Bartlett. Sparse greedy Gaussian process regression. *Advances in neural information processing systems*, 13, 2000.

E. Snelson and Z. Ghahramani. Sparse Gaussian processes using pseudo-inputs. *Advances in neural information processing systems*, 18, 2005.

M. L. Stein, Z. Chi, and L. J. Welty. Approximating likelihoods for large spatial data sets. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 66(2):275–296, 2004.

B. Sudret, S. Marelli, and J. Wiart. Surrogate models for uncertainty quantification: An overview. In *2017 11th European conference on antennas and propagation (EUCAP)*, pages 793–797. IEEE, 2017.

G. Sun and S. Wang. A review of the artificial neural network surrogate modeling in aerodynamic design. *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering*, 233 (16):5863–5872, 2019.

S. Surjanovic and D. Bingham. Virtual library of simulation experiments: test functions and datasets. *Simon Fraser University, Burnaby, BC, Canada, accessed May*, 13:2015, 2013.

R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996.

M. Tipping. The relevance vector machine. *Advances in neural information processing systems*, 12, 1999.

V. Tresp. A bayesian committee machine. *Neural computation*, 12(11):2719–2741, 2000.

R. K. Tripathy and I. Bilionis. Deep uq: Learning deep neural network surrogate models for high dimensional uncertainty quantification. *Journal of computational physics*, 375:565–588, 2018.

K. Ushey, J. Allaire, and Y. Tang. *reticulate: Interface to 'Python'*, 2025. URL `https://CRAN.R-project.org/package=reticulate`. R package version 1.43.0.

A. V. Vecchia. Estimation and model identification for continuous spatial processes. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 50(2):297–312, 1988.

A. Visokay, E. Hock, K. Hoffman, and T. H. McCormick. When you come to a fork in the road, take it: the rashomon effect for social scientists. 2025.

E. Volkova, B. Iooss, and F. Van Dorpe. Global sensitivity analysis for a numerical model of radionuclide migration from the rrc "kurchatov institute" radwaste disposal site. *Stochastic Environmental Research and Risk Assessment*, 22(1):17–31, 2008.

V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic learning in a random world*. Springer, 2005.

D. J. Walters, A. Biswas, E. C. Lawrence, D. C. Francom, D. J. Luscher, D. A. Fredenburg, K. R. Moran, C. M. Sweeney, R. L. Sandberg, J. P. Ahrens, et al. Bayesian calibration of strength parameters using hydrocode simulations of symmetric impact shock experiments of Al-5083. *Journal of Applied Physics*, 124(20), 2018.

C. Wang, X. Qiang, M. Xu, and T. Wu. Recent advances in surrogate modeling methods for uncertainty quantification and propagation. *Symmetry*, 14(6):1219, 2022.

N. Wiener. The homogeneous chaos. *American Journal of Mathematics*, 60(4):897–936, 1938.

D. H. Wolpert. The supervised learning no-free-lunch theorems. *Soft computing and industry: Recent applications*, pages 25–42, 2002.

N. Wycoff, M. Binois, and S. M. Wild. Sequential learning of active subspaces. *Journal of Computational and Graphical Statistics*, 30(4):1224–1237, 2021.

R. Xin, C. Zhong, Z. Chen, T. Takagi, M. Seltzer, and C. Rudin. Exploring the whole rashomon set of sparse decision trees. *Advances in neural information processing systems*, 35:14071–14084, 2022.

D. Xiu and G. E. Karniadakis. The wiener–askey polynomial chaos for stochastic differential equations. *SIAM journal on scientific computing*, 24(2):619–644, 2002.

J. C. Yannotty, T. J. Santner, R. J. Furnstahl, and M. T. Pratola. Model mixing using Bayesian additive regression trees. *Technometrics*, 66(2):196–207, 2024.

S. Zamanian, J. Hur, and A. Shafieezadeh. Significant variables for leakage and collapse of buried concrete sewer pipes: A global sensitivity analysis via bayesian additive regression trees and sobol'indices. *Structure and infrastructure engineering*, 17(5):676–688, 2021.

Q. Zhang, P. Chien, Q. Liu, L. Xu, and Y. Hong. Mixed-input Gaussian process emulators for computer experiments with a large number of categorical levels. *Journal of Quality Technology*, 53(4):410–420, 2021.

Y. Zhou, Z. Lu, and K. Cheng. A new surrogate modeling method combining polynomial chaos expansion and Gaussian kernel in a sparse bayesian learning framework. *International Journal for Numerical Methods in Engineering*, 120(4):498–516, 2019.

X. Zhu and B. Sudret. Stochastic polynomial chaos expansions to emulate stochastic simulators. *International Journal for Uncertainty Quantification*, 13(2), 2023.